Ex. No. 1	Working with Widgets and Layouts
Date of Exercise	09.08.2026
GitHub Code Link (Keep as public)	https://github.com/henrysam/FlutterLaboratory

Aim

To design and develop a Flutter application that displays a list of fruit items using Row, Column, Container, Card and ListView widgets, and to create reusable custom widgets to organize the user interface effectively.

Description

Flutter is a UI framework used to build applications for multiple platforms from a single codebase. In Flutter, the user interface is constructed using widgets. Widgets describe the structure, layout, styling and behaviour of the application.

This experiment implements a simple fruit-list application named “Fresh Fruits”. The home screen contains an application bar, a welcome section, a heading and a scrollable list of fruit items. Each fruit is displayed in a card with an emoji, name, short description and price.

The following widgets are used in this experiment:

- Row: Arranges its child widgets horizontally.
- Column: Arranges its child widgets vertically.
- Container: Provides layout space, padding, dimensions, background colour and decoration.
- Card: Groups related information in an elevated visual panel.
- ListView.builder: Builds a scrollable list of items efficiently from the fruit data.
- Expanded: Allows the list to occupy the remaining available vertical space.

A Fruit class is used as a simple data model to store each fruit’s name, emoji, description and price. FruitCard is a reusable custom StatelessWidget. The same FruitCard layout is reused for every fruit by passing a different Fruit object to it. This reduces code duplication and makes the interface easier to maintain.

The application is executed in Google Chrome using Flutter Web. The ListView can be scrolled to view all the fruit entries.

Program

File: lib/main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

// Main application widget
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Fresh Fruits',
      theme: ThemeData(
        colorSchemeSeed: Colors.green,
        useMaterial3: true,
      ),
      home: const FruitHomePage(),
    );
  }
}

// Fruit data model
class Fruit {
  final String name;
  final String emoji;
  final String description;
  final String price;

  const Fruit({
    required this.name,
    required this.emoji,
    required this.description,
    required this.price,
  });
}

// Main home page
class FruitHomePage extends StatelessWidget {
  const FruitHomePage({super.key});

  // List of fruit items
  static const List<Fruit> fruits = [
    Fruit(
      name: 'Apple',
      emoji: '',
      description: 'Fresh and juicy red apples',
      price: '₹120 / kg',
    ),
    Fruit(
      name: 'Banana',
      emoji: '',
      description: 'Sweet and ripe bananas',
      price: '₹60 / dozen',
    ),
    Fruit(
```

```
    name: 'Orange',
    emoji: '',
    description: 'Rich in Vitamin C',
    price: '₹80 / kg',
  ),
  Fruit(
    name: 'Strawberry',
    emoji: '',
    description: 'Fresh and delicious strawberries',
    price: '₹200 / box',
  ),
  Fruit(
    name: 'Grapes',
    emoji: '',
    description: 'Sweet seedless grapes',
    price: '₹100 / kg',
  ),
  Fruit(
    name: 'Watermelon',
    emoji: '',
    description: 'Refreshing summer fruit',
    price: '₹40 / kg',
  ),
];

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text(
        'Fresh Fruits ',
        style: TextStyle(fontWeight: FontWeight.bold),
      ),
      backgroundColor: Colors.green,
      foregroundColor: Colors.white,
    ),

    // Column arranges the header and list vertically
    body: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        // Welcome section using Container
        Container(
          width: double.infinity,
          padding: const EdgeInsets.all(20),
          color: Colors.green.shade50,
          child: const Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'Welcome!',
                style: TextStyle(
                  fontSize: 25,
                  fontWeight: FontWeight.bold,
                ),
              ),
              SizedBox(height: 5),
              Text(
                'Choose your favourite fresh fruits.',
                style: TextStyle(fontSize: 15),
              ),
            ],
          ),
        ),
```

```
          // Section heading
          const Padding(
            padding: EdgeInsets.fromLTRB(16, 20, 16, 10),
            child: Text(
              'Available Fruits',
              style: TextStyle(
                fontSize: 21,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),

          // ListView displays all fruit cards
          Expanded(
            child: ListView.builder(
              padding: const EdgeInsets.symmetric(horizontal: 12),
              itemCount: fruits.length,
              itemBuilder: (context, index) {
                return FruitCard(fruit: fruits[index]);
              },
            ),
          ),
        ],
      ),
    );
  }
}

// Reusable custom widget for displaying a fruit
class FruitCard extends StatelessWidget {
  final Fruit fruit;

  const FruitCard({
    super.key,
    required this.fruit,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 3,
      margin: const EdgeInsets.symmetric(vertical: 7),
      child: Padding(
        padding: const EdgeInsets.all(14),

        // Row arranges fruit details horizontally
        child: Row(
          children: [
            // Container displays the fruit emoji
            Container(
              width: 65,
              height: 65,
              alignment: Alignment.center,
              decoration: BoxDecoration(
                color: Colors.green.shade50,
                borderRadius: BorderRadius.circular(12),
              ),
              child: Text(
                fruit.emoji,
                style: const TextStyle(fontSize: 35),
              ),
```

```
          ),

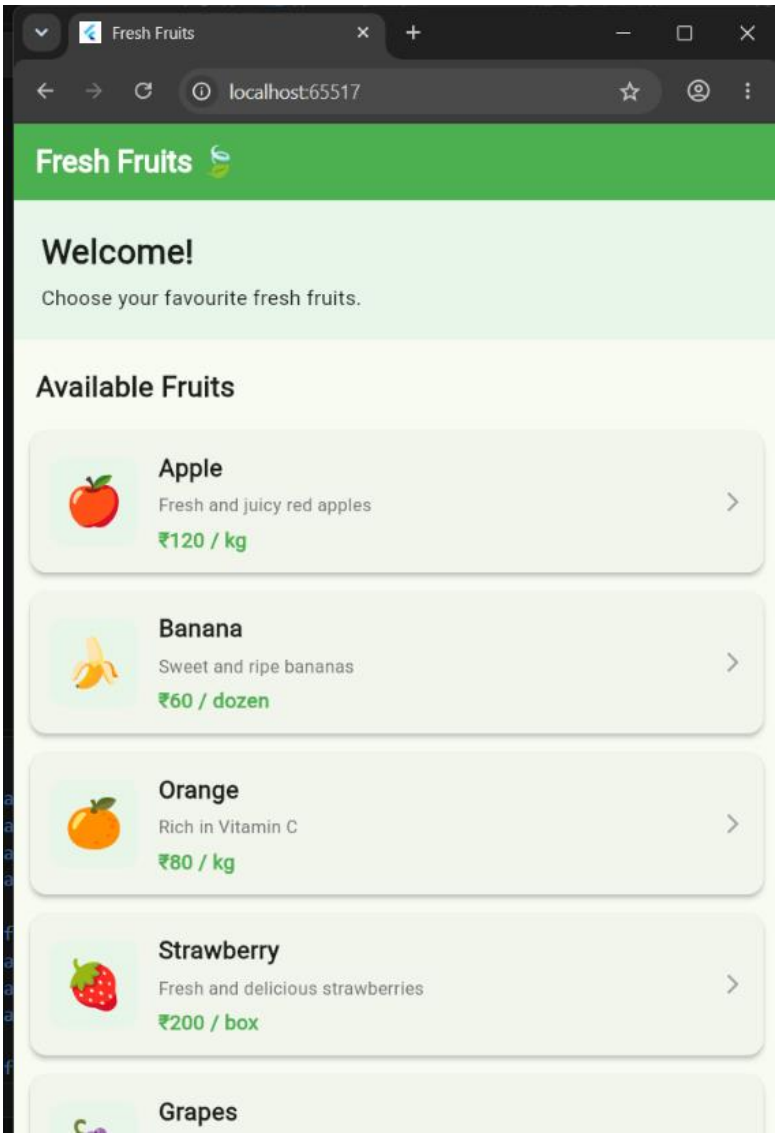
          const SizedBox(width: 15),

          // Column arranges fruit details vertically
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  fruit.name,
                  style: const TextStyle(
                    fontSize: 18,
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 5),
                Text(
                  fruit.description,
                  style: TextStyle(
                    fontSize: 13,
                    color: Colors.grey.shade600,
                  ),
                ),
                const SizedBox(height: 5),
                Text(
                  fruit.price,
                  style: const TextStyle(
                    fontSize: 15,
                    color: Colors.green,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ],
            ),
          ),

          const Icon(
            Icons.arrow_forward_ios,
            size: 17,
            color: Colors.grey,
          ),
        ],
      ),
    ),
  );
 }
}
```

Output Screenshots

The following screenshot shows the Fresh Fruits application running successfully in Google Chrome. The screen displays the application bar, welcome section and fruit cards. The list can be scrolled to view the remaining fruits.



Result

Thus, the Flutter application “Fresh Fruits” was successfully designed and executed. The application displays fruit items using Row, Column, Container, Card and ListView widgets. A reusable custom widget named FruitCard was implemented to display each fruit consistently. The application was run successfully in Google Chrome using Flutter Web.

Ex. No. 2	Adaptive and Responsive UI Design
Date of Exercise	31 October 2026
GitHub Code Link (Keep as public)	https://github.com/henrysam/FlutterLaboratory

24CS2016 – BUILDING APPLICATIONS USING FLUTTER

Aim

To design and implement a responsive Flutter application for an E-Commerce Product Catalog by creating reusable custom product widgets and applying responsive layout techniques so that the user interface adapts to different screen sizes.

Description

Responsive UI design is the process of arranging and resizing interface elements so that an application remains usable on devices with different screen dimensions. In Flutter, responsive layouts can be created by checking the available space and choosing suitable layout properties.

This experiment implements an e-commerce product catalog named “ShopEase”. The screen contains an application bar, a search field, a promotional welcome banner, a product section and a grid of product cards. Each product card displays an emoji representing the product, its name, category, price, rating and an Add to Cart button.

The main widgets and techniques used are:

- LayoutBuilder: Reads the maximum width available to the page so that the layout can respond to the screen size.
- Responsive breakpoints: Uses two columns below 600 pixels, three columns from 600 to below 900 pixels, and four columns at 900 pixels or above.
- GridView.builder: Creates the product cards in a scrollable grid using the selected number of columns.
- ProductCard: A reusable custom StatelessWidget that displays the details of one product.
- Product model: Stores each product’s name, category, emoji, price, rating and background colour.
- ConstrainedBox: Limits the content width to 1200 pixels on wide screens.
- Expanded and Flexible: Allocate available space and help prevent content overflow.
- setState(): Updates the cart count when the Add to Cart button is pressed.

The LayoutBuilder checks the available width and selects the grid configuration. For widths below 600 pixels, two product columns are displayed. For widths from 600 to below 900 pixels, three columns are displayed. For widths of 900 pixels or more, four columns are displayed. The product card aspect ratio is also adjusted for these width ranges.

The application is tested in Google Chrome at both desktop and narrow/mobile-sized browser widths. This demonstrates that the same Flutter application can adapt its product grid to different available screen sizes.

Program

File: lib/main.dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const ShopEaseApp());
}

// MAIN APPLICATION
class ShopEaseApp extends StatelessWidget {
  const ShopEaseApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'ShopEase',
      theme: ThemeData(
        colorSchemeSeed: Colors.deepPurple,
        useMaterial3: true,
      ),
      home: const ProductHomePage(),
    );
  }
}

// PRODUCT DATA MODEL
class Product {
  final String name;
  final String category;
  final String emoji;
  final double price;
  final double rating;
  final Color color;

  const Product({
    required this.name,
    required this.category,
    required this.emoji,
    required this.price,
    required this.rating,
    required this.color,
  });
}

// MAIN PRODUCT PAGE
class ProductHomePage extends StatefulWidget {
  const ProductHomePage({super.key});

  @override
  State<ProductHomePage> createState() => _ProductHomePageState();
}

class _ProductHomePageState extends State<ProductHomePage> {
  // PRODUCT LIST
  final List<Product> products = const [
    Product(
      name: 'Wireless Headphones',
      category: 'Electronics',
      emoji: '',
      price: 1499,
      rating: 4.8,
      color: Color(0xFFEDE7F6),
```

```
  ),
  Product(
    name: 'Smart Watch',
    category: 'Electronics',
    emoji: '⌚',
    price: 2499,
    rating: 4.6,
    color: Color(0xFFE3F2FD),
  ),
  Product(
    name: 'Running Shoes',
    category: 'Fashion',
    emoji: '👟',
    price: 1999,
    rating: 4.5,
    color: Color(0xFFFFF3E0),
  ),
  Product(
    name: 'Backpack',
    category: 'Accessories',
    emoji: '🎒',
    price: 999,
    rating: 4.4,
    color: Color(0xFFE8F5E9),
  ),
  Product(
    name: 'Camera',
    category: 'Electronics',
    emoji: '📷',
    price: 5999,
    rating: 4.9,
    color: Color(0xFFFCE4EC),
  ),
  Product(
    name: 'Sunglasses',
    category: 'Fashion',
    emoji: '🕶️',
    price: 799,
    rating: 4.3,
    color: Color(0xFFFFF9C4),
  ),
  Product(
    name: 'Smartphone',
    category: 'Electronics',
    emoji: '📱',
    price: 12999,
    rating: 4.7,
    color: Color(0xFFE0F7FA),
  ),
  Product(
    name: 'Wrist Watch',
    category: 'Accessories',
    emoji: '⌚',
    price: 1799,
    rating: 4.5,
    color: Color(0xFFF3E5F5),
  ),
];

int cartCount = 0;

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F7FB),

    // APP BAR
```

```
appBar: AppBar(
  backgroundColor: Colors.deepPurple,
  foregroundColor: Colors.white,
  title: const Text(
    'ShopEase',
    style: TextStyle(fontWeight: FontWeight.bold),
  ),
  actions: [
    Padding(
      padding: const EdgeInsets.only(right: 20),
      child: Row(
        children: [
          const Icon(Icons.shopping_cart_outlined),
          const SizedBox(width: 5),
          Text('$cartCount'),
        ],
      ),
    ),
  ],
),

// RESPONSIVE BODY
body: LayoutBuilder(
  builder: (context, constraints) {
    final width = constraints.maxWidth;


    // RESPONSIVE BREAKPOINTS
    int columns;


    if (width < 600) {
      columns = 2;
    } else if (width < 900) {
      columns = 3;
    } else {
      columns = 4;
    }


    // DIFFERENT CARD SIZES
    double cardRatio;


    if (width < 600) {
      cardRatio = 0.62;
    } else if (width < 900) {
      cardRatio = 0.72;
    } else {
      cardRatio = 0.78;
    }


    return Center(
      child: ConstrainedBox(
        constraints: const BoxConstraints(
          maxWidth: 1200,
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // SEARCH BAR
            Padding(
              padding: const EdgeInsets.all(16),
              child: TextField(
                decoration: InputDecoration(
                  hintText: 'Search products...',
                  prefixIcon: const Icon(Icons.search),
                  filled: true,
                  fillColor: Colors.white,
                  border: OutlineInputBorder(
```

```
        borderRadius: BorderRadius.circular(15),
        borderSide: BorderSide.none,
      ),
    ),
  ),
),

// WELCOME BANNER
Container(
  width: double.infinity,
  margin: const EdgeInsets.symmetric(
    horizontal: 16,
  ),
  padding: const EdgeInsets.all(18),
  decoration: BoxDecoration(
    color: Colors.deepPurple.shade50,
    borderRadius: BorderRadius.circular(15),
  ),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        'Discover Amazing Products!',
        style: TextStyle(
          fontSize: width < 600 ? 18 : 23,
          fontWeight: FontWeight.bold,
          color: Colors.deepPurple,
        ),
      ),
      const SizedBox(height: 5),
      const Text(
        'Find the best products at great prices.',
      ),
    ],
  ),
),

// SECTION TITLE
const Padding(
  padding: EdgeInsets.fromLTRB(16, 20, 16, 12),
  child: Text(
    'Popular Products',
    style: TextStyle(
      fontSize: 21,
      fontWeight: FontWeight.bold,
    ),
  ),
),

// RESPONSIVE PRODUCT GRID
Expanded(
  child: GridView.builder(
    padding: const EdgeInsets.symmetric(
      horizontal: 16,
      vertical: 5,
    ),
    itemCount: products.length,
    gridDelegate:
        SliverGridDelegateWithFixedCrossAxisCount(
      crossAxisCount: columns,
      crossAxisSpacing: 14,
      mainAxisSpacing: 14,
      childAspectRatio: cardRatio,
    ),
    itemBuilder: (context, index) {
      return ProductCard(
        product: products[index],
```

```
                        onAddToCart: () {
                          setState(() {
                            cartCount++;
                          });

                          ScaffoldMessenger.of(context).showSnackBar(
                            SnackBar(
                              content: Text(
                                '${products[index].name} added to cart!',
                              ),
                              duration: const Duration(seconds: 1),
                            ),
                          );
                        },
                      );
                    },
                  ),
                ),
              ],
            ),
          ),
        );
      },
    ),
  );
  }
}

// REUSABLE CUSTOM PRODUCT WIDGET
class ProductCard extends StatelessWidget {
  final Product product;
  final VoidCallback onAddToCart;

  const ProductCard({
    super.key,
    required this.product,
    required this.onAddToCart,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      color: Colors.white,
      elevation: 3,
      clipBehavior: Clip.antiAlias,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(15),
      ),
      child: Padding(
        padding: const EdgeInsets.all(10),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // PRODUCT IMAGE AREA
            Expanded(
              child: Container(
                width: double.infinity,
                decoration: BoxDecoration(
                  color: product.color,
                  borderRadius: BorderRadius.circular(12),
                ),
                child: Center(
                  child: Text(
                    product.emoji,
                    style: const TextStyle(fontSize: 55),
                  ),
                ),
```

```
  ),
),

const SizedBox(height: 10),

// PRODUCT NAME
Text(
  product.name,
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
  style: const TextStyle(
    fontSize: 15,
    fontWeight: FontWeight.bold,
  ),
),

const SizedBox(height: 3),

// PRODUCT CATEGORY
Text(
  product.category,
  style: TextStyle(
    color: Colors.grey.shade600,
    fontSize: 12,
  ),
),

const SizedBox(height: 5),

// PRICE AND RATING
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Flexible(
      child: Text(
        '₹${product.price.toStringAsFixed(0)}',
        overflow: TextOverflow.ellipsis,
        style: const TextStyle(
          color: Colors.deepPurple,
          fontSize: 14,
          fontWeight: FontWeight.bold,
        ),
      ),
    ),
    const SizedBox(width: 3),
    const Icon(
      Icons.star,
      color: Colors.amber,
      size: 15,
    ),
    Text(
      product.rating.toString(),
      style: const TextStyle(fontSize: 12),
    ),
  ],
),

const SizedBox(height: 8),

// ADD TO CART BUTTON
SizedBox(
  width: double.infinity,
  height: 35,
  child: ElevatedButton(
```

```
                onPressed: onAddToCart,
                style: ElevatedButton.styleFrom(
                  backgroundColor: Colors.deepPurple,
                  foregroundColor: Colors.white,
                  padding: EdgeInsets.zero,
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(8),
                  ),
                ),
                child: const Text(
                  'Add to Cart',
                  style: TextStyle(fontSize: 12),
                ),
              ),
            ),
          ),
        ],
      ),
    ),
  );
 }
}
```

Output Screenshots

The screenshots below show the ShopEase application running in Google Chrome at two different browser widths. The desktop-sized view displays four product columns, while the narrow/mobile-sized view displays two product columns. This demonstrates the responsive grid behaviour implemented using LayoutBuilder and GridView.builder.

Figure 1: Desktop / wide-screen layout

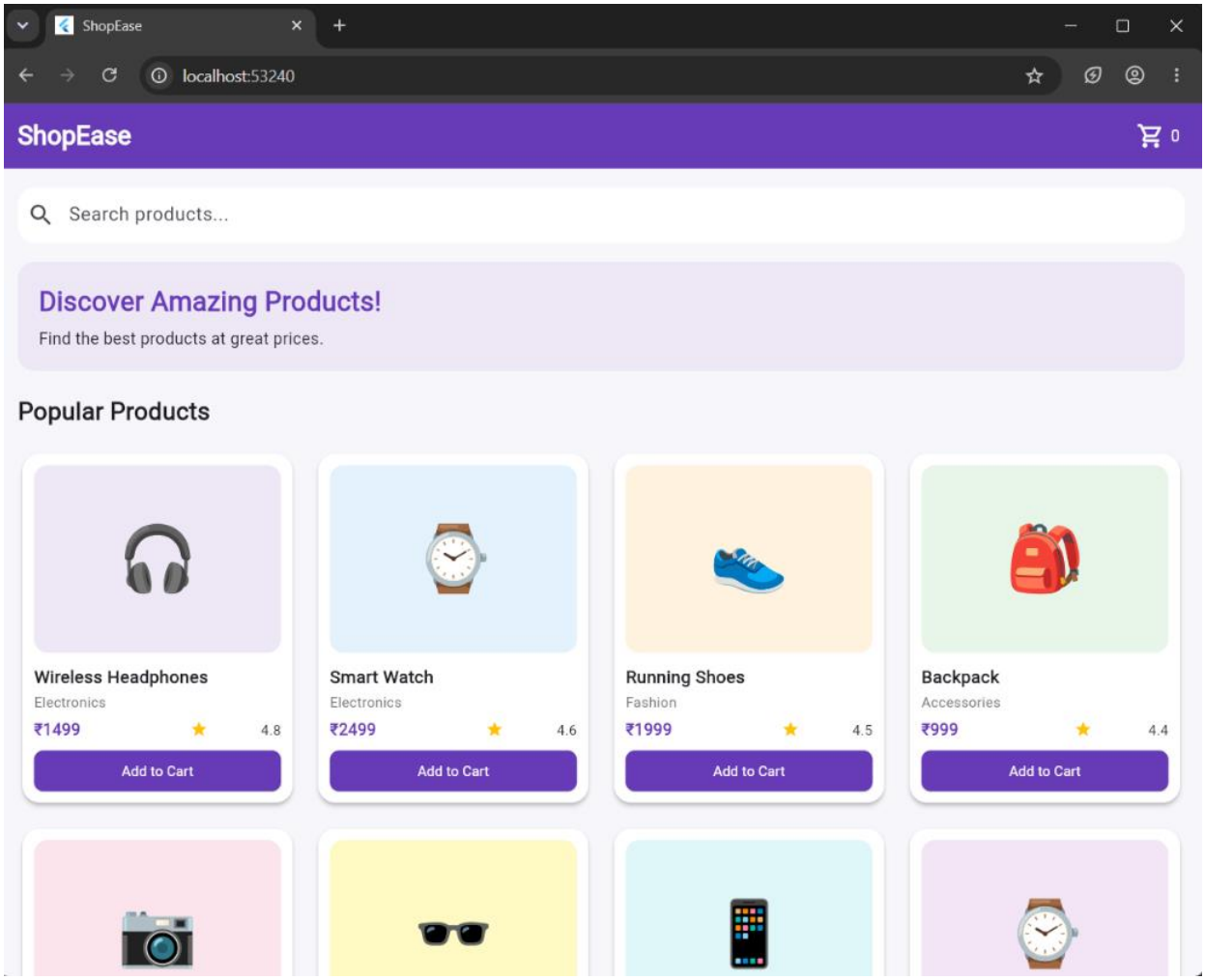
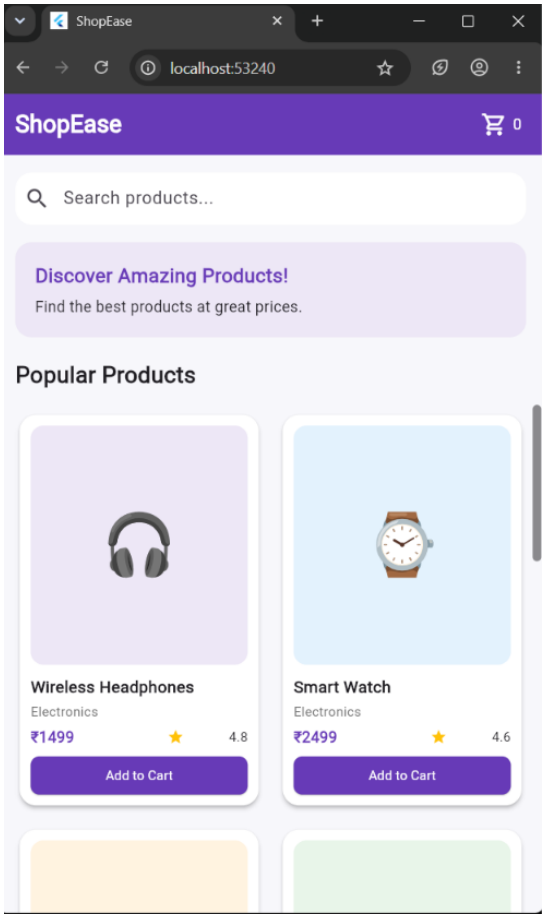


Figure 2: Narrow / mobile-sized layout



Result

Thus, the ShopEase E-Commerce Product Catalog application was successfully designed and implemented using Flutter. Reusable custom product widgets were created to display product information. Responsive layout techniques using LayoutBuilder, screen-width breakpoints and GridView.builder were implemented to adapt the product grid to different screen sizes. The application was tested successfully in Google Chrome using both desktop-sized and narrow/mobile-sized layouts.

Ex. No. 3	Add Interactivity to App – Input and Forms
Date of Exercise	31 October 2026
GitHub Code Link (Keep as public)	https://github.com/henrysam/FlutterLaboratory

24CS2016 – BUILDING APPLICATIONS USING FLUTTER

Aim

To develop an interactive Flutter application that displays a predefined list of products and allows users to search and filter products using input widgets, thereby providing an interactive user experience.

Description

Input widgets allow users to provide information or make selections in an application. Flutter provides widgets such as TextField, ChoiceChip and DropdownButtonFormField to collect user input. Event callbacks respond to user actions, while setState() updates the user interface when the application state changes.

This experiment implements an interactive e-commerce product catalog named “ShopEase Interactive Store”. A predefined list of products is displayed as cards in a grid. Users can enter a product name in the search field, select a category, choose a sorting option and clear the applied filters.

The main widgets and concepts used are:

- TextField: Accepts search text from the user. Its onChanged callback updates the search query as the user types.
- ChoiceChip: Allows the user to select a product category such as All, Electronics, Fashion or Accessories.
- DropdownButtonFormField: Provides sorting choices including default order, price from low to high, price from high to low and rating from high to low.
- TextEditingController: Controls the search field and allows its text to be cleared programmatically.
- setState(): Updates the application state and rebuilds the relevant user interface after search, filter, sort or cart actions.
- List filtering: The product list is filtered by matching the search text and selected category.
- List sorting: The filtered products are sorted according to the selected price or rating option.
- GridView.builder: Displays the resulting products in a scrollable grid.
- ProductCard: A reusable custom widget that displays each product’s emoji, name, category, price, rating and Add to Cart button.

When the user enters text, the application compares the lowercase search query with the lowercase product name. It also checks whether each product belongs to the selected category. Only products satisfying both conditions are displayed. The sorting option then arranges the matching products by price or rating. If no products match, a “No products found!” message is displayed. The Clear Filters button resets the search text, category and sorting option. The Add to Cart button increments the cart count and displays a confirmation message.

The application was executed in Google Chrome. The screenshots show the default product list, the available sorting options and the search result when the user enters “wat”, which returns Smart Watch and Wrist Watch.

Program

File: lib/main.dart

```
import 'package:flutter/material.dart';


void main() {
  runApp(const ShopEaseApp());
}


// MAIN APPLICATION
class ShopEaseApp extends StatelessWidget {
  const ShopEaseApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'ShopEase Interactive Store',
      theme: ThemeData(
        colorSchemeSeed: Colors.deepPurple,
        useMaterial3: true,
      ),
      home: const ProductHomePage(),
    );
  }
}


// PRODUCT MODEL
class Product {
  final String name;
  final String category;
  final String emoji;
  final double price;
  final double rating;
  final Color color;


  const Product({
    required this.name,
    required this.category,
    required this.emoji,
    required this.price,
    required this.rating,
    required this.color,
  });
}


// MAIN PRODUCT PAGE
class ProductHomePage extends StatefulWidget {
  const ProductHomePage({super.key});


  @override
  State<ProductHomePage> createState() => _ProductHomePageState();
}


class _ProductHomePageState extends State<ProductHomePage> {
  // PREDEFINED PRODUCT LIST
  final List<Product> products = const [
    Product(
      name: 'Wireless Headphones',
      category: 'Electronics',
      emoji: '',
      price: 1499,
      rating: 4.8,
      color: Color(0xFFEDE7F6),
    ),
    Product(
      name: 'Smart Watch',
```

```
    category: 'Electronics',
    emoji: '⌚',
    price: 2499,
    rating: 4.6,
    color: Color(0xFFE3F2FD),
  ),
  Product(
    name: 'Running Shoes',
    category: 'Fashion',
    emoji: '👟',
    price: 1999,
    rating: 4.5,
    color: Color(0xFFFFF3E0),
  ),
  Product(
    name: 'Backpack',
    category: 'Accessories',
    emoji: '🎒',
    price: 999,
    rating: 4.4,
    color: Color(0xFFE8F5E9),
  ),
  Product(
    name: 'Camera',
    category: 'Electronics',
    emoji: '📷',
    price: 5999,
    rating: 4.9,
    color: Color(0xFFFCE4EC),
  ),
  Product(
    name: 'Sunglasses',
    category: 'Fashion',
    emoji: '🕶',
    price: 799,
    rating: 4.3,
    color: Color(0xFFFFF9C4),
  ),
  Product(
    name: 'Smartphone',
    category: 'Electronics',
    emoji: '📱',
    price: 12999,
    rating: 4.7,
    color: Color(0xFFE0F7FA),
  ),
  Product(
    name: 'Wrist Watch',
    category: 'Accessories',
    emoji: '⌚',
    price: 1799,
    rating: 4.5,
    color: Color(0xFFF3E5F5),
  ),
];


// INPUT WIDGET VARIABLES
String searchQuery = '';
String selectedCategory = 'All';
String sortOption = 'Default';
int cartCount = 0;


final TextEditingController searchController =
    TextEditingController();


// GET FILTERED PRODUCTS
List<Product> get filteredProducts {
  List<Product> result = products.where((product) {
    final matchesSearch = product.name
        .toLowerCase()
        .contains(searchQuery.toLowerCase());
```

```
    final matchesCategory =
        selectedCategory == 'All' ||
        product.category == selectedCategory;

    return matchesSearch && matchesCategory;
  }).toList();

  // SORT PRODUCTS
  if (sortOption == 'Price: Low to High') {
    result.sort((a, b) => a.price.compareTo(b.price));
  } else if (sortOption == 'Price: High to Low') {
    result.sort((a, b) => b.price.compareTo(a.price));
  } else if (sortOption == 'Rating: High to Low') {
    result.sort((a, b) => b.rating.compareTo(a.rating));
  }

  return result;
}

// CLEAR ALL FILTERS
void clearFilters() {
  setState(() {
    searchQuery = '';
    selectedCategory = 'All';
    sortOption = 'Default';
    searchController.clear();
  });
}

@override
void dispose() {
  searchController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F7FB),

    // APPLICATION BAR
    appBar: AppBar(
      backgroundColor: Colors.deepPurple,
      foregroundColor: Colors.white,
      title: const Text(
        'ShopEase',
        style: TextStyle(fontWeight: FontWeight.bold),
      ),
      actions: [
        Padding(
          padding: const EdgeInsets.only(right: 20),
          child: Row(
            children: [
              const Icon(Icons.shopping_cart_outlined),
              const SizedBox(width: 5),
              Text('$cartCount'),
            ],
          ),
        ),
      ],
    ),

    // APPLICATION BODY
    body: LayoutBuilder(
      builder: (context, constraints) {
        final isWide = constraints.maxWidth >= 600;
```

```
return Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    // SEARCH INPUT
    Padding(
      padding: const EdgeInsets.all(16),
      child: TextField(
        controller: searchController,
        onChanged: (value) {
          setState(() {
            searchQuery = value;
          });
        },
        decoration: InputDecoration(
          hintText: 'Search products...',
          prefixIcon: const Icon(Icons.search),
          suffixIcon: IconButton(
            icon: const Icon(Icons.clear),
            onPressed: () {
              setState(() {
                searchController.clear();
                searchQuery = '';
              });
            },
          ),
          filled: true,
          fillColor: Colors.white,
          border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(15),
            borderSide: BorderSide.none,
          ),
        ),
      ),
    ),

    // FILTER SECTION
    Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text(
            'Filter by Category',
            style: TextStyle(
              fontWeight: FontWeight.bold,
              fontSize: 17,
            ),
          ),
          const SizedBox(height: 10),

          // CATEGORY FILTER CHIPS
          Wrap(
            spacing: 8,
            runSpacing: 5,
            children: [
              'All',
              'Electronics',
              'Fashion',
              'Accessories',
            ].map((category) {
              return ChoiceChip(
                label: Text(category),
                selected: selectedCategory == category,
                onSelected: (selected) {
                  setState(() {
                    selectedCategory =
                        selected ? category : 'All';
                  });
                },
              );
            }).toList(),
```

```
        ),

        const SizedBox(height: 12),

        // SORTING DROPDOWN
        Row(
          children: [
            const Text(
              'Sort by:',
              style: TextStyle(
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: DropdownButtonFormField<String>(
                initialValue: sortOption,
                decoration: InputDecoration(
                  filled: true,
                  fillColor: Colors.white,
                  contentPadding:
                      const EdgeInsets.symmetric(
                    horizontal: 12,
                    vertical: 8,
                  ),
                  border: OutlineInputBorder(
                    borderRadius:
                        BorderRadius.circular(12),
                    borderSide: BorderSide.none,
                  ),
                ),
                items: [
                  'Default',
                  'Price: Low to High',
                  'Price: High to Low',
                  'Rating: High to Low',
                ].map((option) {
                  return DropdownMenuItem(
                    value: option,
                    child: Text(
                      option,
                      style: const TextStyle(
                        fontSize: 13,
                      ),
                    ),
                  );
                }).toList(),
                onChanged: (value) {
                  setState(() {
                    sortOption = value!;
                  });
                },
              ),
            ),
          ],
        ),

        // CLEAR FILTER BUTTON
        Align(
          alignment: Alignment.centerRight,
          child: TextButton.icon(
            onPressed: clearFilters,
            icon: const Icon(Icons.refresh),
            label: const Text('Clear Filters'),
          ),
        ),
      ],
    ),
  ),

  // PRODUCT COUNT
```

```
        Padding(
          padding: const EdgeInsets.symmetric(
            horizontal: 16,
            vertical: 5,
          ),
          child: Text(
            '${filteredProducts.length} Products Found',
            style: const TextStyle(
              fontSize: 18,
              fontWeight: FontWeight.bold,
            ),
          ),
        ),

        // FILTERED PRODUCT LIST
        Expanded(
          child: filteredProducts.isEmpty
              ? const Center(
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      Icon(
                        Icons.search_off,
                        size: 60,
                        color: Colors.grey,
                      ),
                      SizedBox(height: 10),
                      Text(
                        'No products found!',
                        style: TextStyle(fontSize: 18),
                      ),
                    ],
                  ),
                )
              : GridView.builder(
                  padding: const EdgeInsets.all(12),
                  itemCount: filteredProducts.length,
                  gridDelegate:
                      SliverGridDelegateWithFixedCrossAxisCount(
                    crossAxisCount: isWide ? 3 : 2,
                    crossAxisSpacing: 12,
                    mainAxisSpacing: 12,
                    childAspectRatio: 0.68,
                  ),
                  itemBuilder: (context, index) {
                    final product = filteredProducts[index];

                    return ProductCard(
                      product: product,
                      onAddToCart: () {
                        setState(() {
                          cartCount++;
                        });

                        ScaffoldMessenger.of(context)
                            .showSnackBar(
                          SnackBar(
                            content: Text(
                              '${product.name} added to cart!',
                            ),
                            duration:
                                const Duration(seconds: 1),
                          ),
                        );
                      },
                    );
                  },
                ),
        ),
      ],
    );
  },
```

```
      ),
    );
  }
}

// REUSABLE PRODUCT CARD
class ProductCard extends StatelessWidget {
  final Product product;
  final VoidCallback onAddToCart;


  const ProductCard({
    super.key,
    required this.product,
    required this.onAddToCart,
  });


  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 3,
      color: Colors.white,
      clipBehavior: Clip.antiAlias,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(15),
      ),
      child: Padding(
        padding: const EdgeInsets.all(10),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // PRODUCT IMAGE
            Expanded(
              child: Container(
                width: double.infinity,
                decoration: BoxDecoration(
                  color: product.color,
                  borderRadius: BorderRadius.circular(12),
                ),
                child: Center(
                  child: Text(
                    product.emoji,
                    style: const TextStyle(fontSize: 50),
                  ),
                ),
              ),
            ),


            const SizedBox(height: 8),


            // PRODUCT NAME
            Text(
              product.name,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: const TextStyle(
                fontSize: 14,
                fontWeight: FontWeight.bold,
              ),
            ),


            const SizedBox(height: 4),


            // CATEGORY
            Text(
              product.category,
              style: const TextStyle(
                color: Colors.grey,
                fontSize: 12,
              ),
```

```
          ),

          const SizedBox(height: 5),

          // PRICE AND RATING
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Text(
                '₹${product.price.toStringAsFixed(0)}',
                style: const TextStyle(
                  color: Colors.deepPurple,
                  fontWeight: FontWeight.bold,
                ),
              ),
              Row(
                children: [
                  const Icon(
                    Icons.star,
                    color: Colors.amber,
                    size: 15,
                  ),
                  Text(
                    product.rating.toString(),
                    style: const TextStyle(fontSize: 12),
                  ),
                ],
              ),
            ],
          ),

          const SizedBox(height: 8),

          // ADD TO CART BUTTON
          SizedBox(
            width: double.infinity,
            height: 35,
            child: ElevatedButton(
              onPressed: onAddToCart,
              style: ElevatedButton.styleFrom(
                backgroundColor: Colors.deepPurple,
                foregroundColor: Colors.white,
                padding: EdgeInsets.zero,
              ),
              child: const Text(
                'Add to Cart',
                style: TextStyle(fontSize: 12),
              ),
            ),
          ),
        ],
      ),
    ),
  );
  }
}
```

Output Screenshots

The following screenshots show the ShopEase Interactive Store running in Google Chrome. Figure 1 shows the default product list and the available category filters. Figure 2 shows the sorting dropdown with the available sorting choices. Figure 3 shows the search operation for the text “wat”, which displays two matching products: Smart Watch and Wrist Watch.

Figure 1: Default product list with category filters

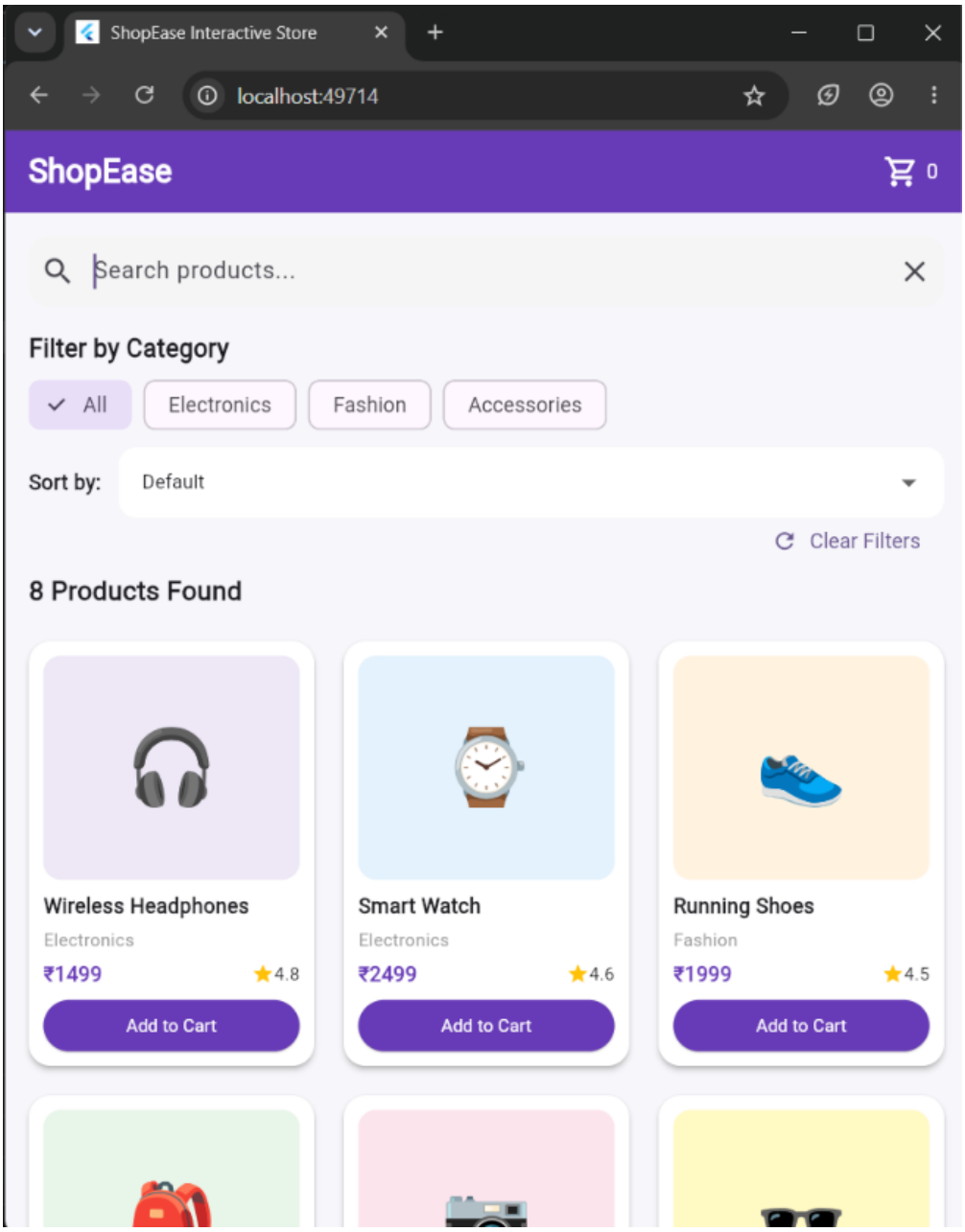
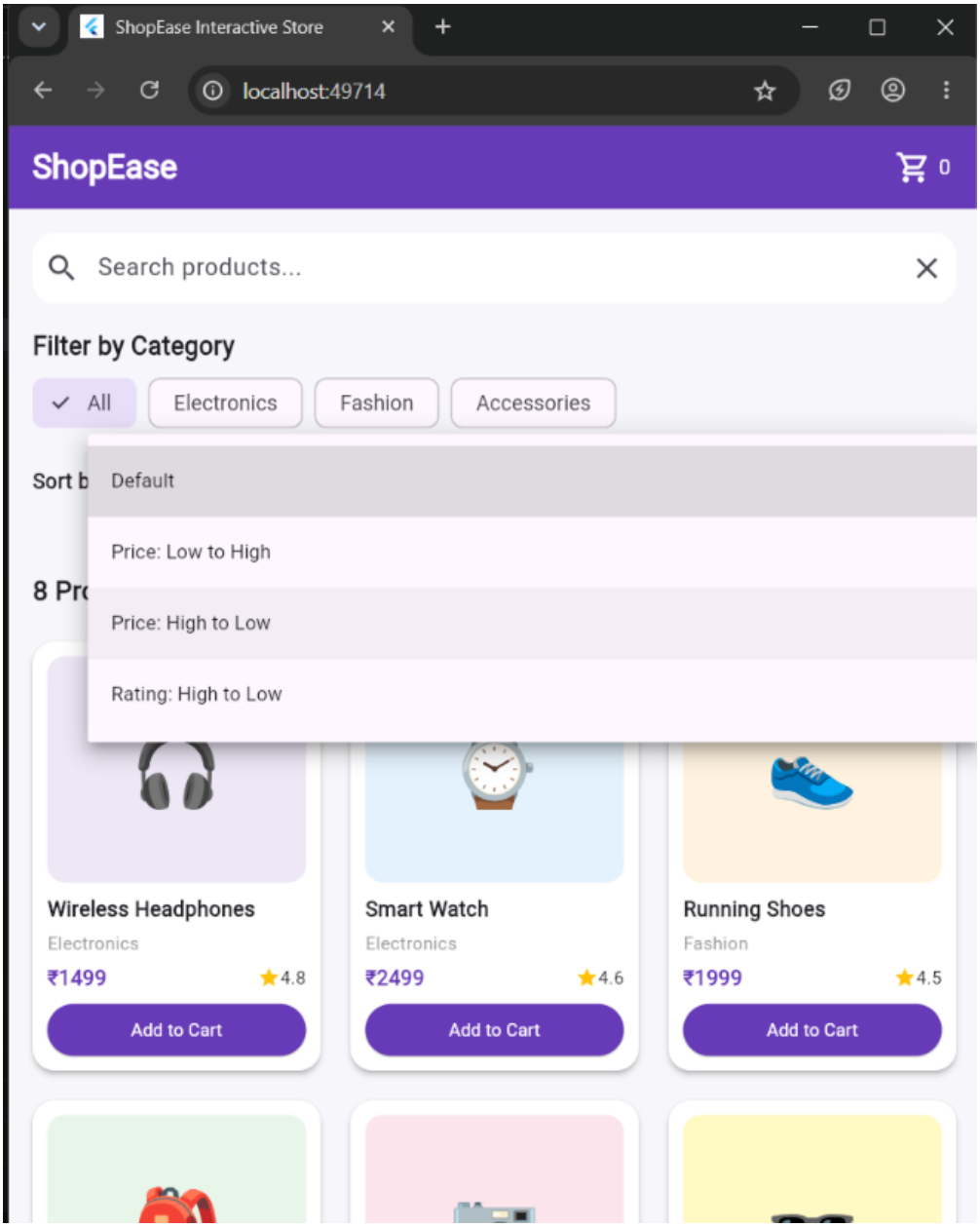
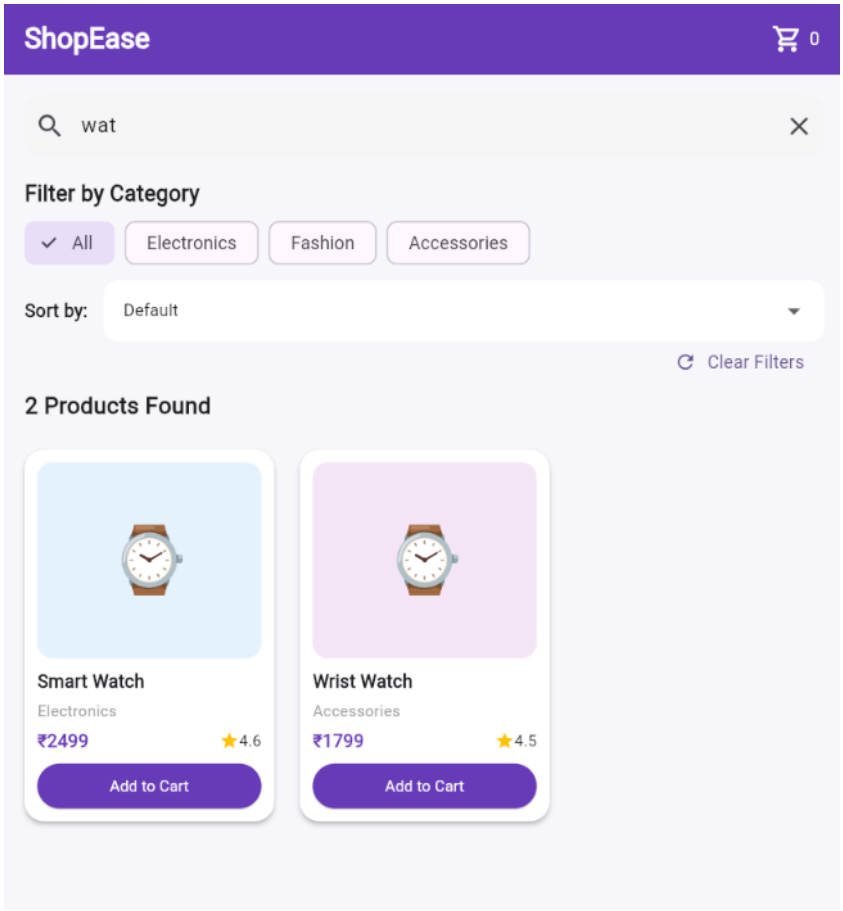


Figure 2: Sorting options displayed in the dropdown


ShopEase Interactive Store
localhost:49714
ShopEase
0
Search products...
Filter by Category
All
Electronics
Fashion
Accessories
Sort b
Default
Price: Low to High
Price: High to Low
Rating: High to Low
8 Pr
Wireless Headphones
Electronics
₹1499
4.8
Add to Cart
Smart Watch
Electronics
₹2499
4.6
Add to Cart
Running Shoes
Fashion
₹1999
4.5
Add to Cart

Output Screenshots (Continued)

Figure 3: Search result for the query "wat"



Result

Thus, the interactive e-commerce product catalog application "ShopEase Interactive Store" was successfully developed using Flutter. The application displays a predefined product list and allows users to search products, filter them by category and sort them by price or rating using input widgets. The Clear Filters option resets the selections, and the Add to Cart action updates the cart count. The application was executed successfully in Google Chrome.

Ex. No. 4	Form Validation using Stateful Widgets
Date of Exercise	31 October 2026
GitHub Code Link (Keep as public)	https://github.com/henrysam/FlutterLaboratory

24CS2016 – BUILDING APPLICATIONS USING FLUTTER

Aim

To design and implement a Car Mileage Calculator using Flutter Forms, TextFormField, input widgets and Stateful Widgets. The application validates the user inputs and dynamically displays the calculated vehicle mileage.

Description

Flutter provides form-related widgets that can collect and validate user input. A Form widget groups input fields, while TextFormField accepts text and supports validation through its validator property. A GlobalKey<FormState> is used to access the form state and validate the fields.

This experiment implements a Car Mileage Calculator. The user enters the distance travelled in kilometres and the fuel consumed in litres. After validating both values, the application calculates the mileage and displays the result in kilometres per litre.

The main widgets and concepts used are:

- StatefulWidget: Maintains changing application data, such as the calculated mileage.
- Form: Groups the input fields and provides a common validation mechanism.
- GlobalKey<FormState>: Provides access to the form state and invokes validation.
- TextFormField: Accepts distance and fuel values and displays validation messages.
- TextEditingController: Reads and clears the values entered by the user.
- TextInputType.numberWithOptions: Displays a numeric keyboard and permits decimal input.
- validator: Checks for empty, non-numeric, zero, negative and non-finite values.
- setState(): Updates the mileage result dynamically after a valid calculation.
- SingleChildScrollView: Allows the form to be scrolled on smaller screens.

The mileage is calculated using the formula: Mileage = Distance Travelled / Fuel Consumed. For example, if a vehicle travels 300 km using 10 litres of fuel, its mileage is 300 / 10 = 30 km/l. The calculation is performed only when both fields pass validation. Empty or invalid values produce an appropriate error message. The Reset button clears the input fields and removes the displayed result.

The application is executed in Google Chrome. In the attached output screenshot, the distance is 300 km and the fuel consumed is 10 litres. The application displays the calculated mileage as 30.00 km/l.

Program

File: lib/main.dart

```
import 'package:flutter/material.dart';


void main() {
  runApp(const MileageApp());
}


// MAIN APPLICATION
class MileageApp extends StatelessWidget {
  const MileageApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Car Mileage Calculator',
      theme: ThemeData(
        colorSchemeSeed: Colors.green,
        useMaterial3: true,
      ),
      home: const MileageHomePage(),
    );
  }
}


// MAIN STATEFUL WIDGET
class MileageHomePage extends StatefulWidget {
  const MileageHomePage({super.key});


  @override
  State<MileageHomePage> createState() =>
      _MileageHomePageState();
}


class _MileageHomePageState extends State<MileageHomePage> {
  // FORM KEY
  final _formKey = GlobalKey<FormState>();


  // INPUT CONTROLLERS
  final TextEditingController distanceController =
      TextEditingController();


  final TextEditingController fuelController =
      TextEditingController();


  // MILEAGE RESULT
  double? mileage;


  // CALCULATE MILEAGE
  void calculateMileage() {
    if (_formKey.currentState!.validate()) {
      final double distance =
          double.parse(distanceController.text);


      final double fuel =
          double.parse(fuelController.text);


      setState(() {
        mileage = distance / fuel;
      });


      FocusScope.of(context).unfocus();
```

```
  }
}

// RESET ALL INPUTS
void resetCalculator() {
  setState(() {
    distanceController.clear();
    fuelController.clear();
    mileage = null;
  });

  _formKey.currentState!.reset();
  FocusScope.of(context).unfocus();
}

@override
void dispose() {
  distanceController.dispose();
  fuelController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF4F7F4),

    // APP BAR
    appBar: AppBar(
      title: const Text(
        'Car Mileage Calculator',
        style: TextStyle(fontWeight: FontWeight.bold),
      ),
      backgroundColor: Colors.green,
      foregroundColor: Colors.white,
      centerTitle: true,
    ),

    // APPLICATION BODY
    body: Center(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: ConstrainedBox(
          constraints: const BoxConstraints(
            maxWidth: 500,
          ),
          child: Card(
            elevation: 4,
            color: Colors.white,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(20),
            ),
            child: Padding(
              padding: const EdgeInsets.all(24),
              child: Form(
                key: _formKey,
                child: Column(
                  crossAxisAlignment:
                      CrossAxisAlignment.stretch,
                  children: [
                    // HEADER ICON
                    const Icon(
                      Icons.directions_car,
                      size: 75,
                      color: Colors.green,
                    ),

                    const SizedBox(height: 12),
```

```
const Text(
  'Calculate Your Mileage',
  textAlign: TextAlign.center,
  style: TextStyle(
    fontSize: 23,
    fontWeight: FontWeight.bold,
  ),
),

const SizedBox(height: 8),

Text(
  'Enter your travel details to calculate '
  'your vehicle mileage.',
  textAlign: TextAlign.center,
  style: TextStyle(
    fontSize: 14,
    color: Colors.grey.shade600,
  ),
),

const SizedBox(height: 30),

// DISTANCE INPUT
const Text(
  'Distance Travelled (km)',
  style: TextStyle(
    fontWeight: FontWeight.bold,
    fontSize: 15,
  ),
),

const SizedBox(height: 8),

TextFormField(
  controller: distanceController,
  keyboardType:
      const TextInputType.numberWithOptions(
    decimal: true,
  ),
  decoration: InputDecoration(
    hintText: 'Enter distance in km',
    prefixIcon: const Icon(
      Icons.route,
      color: Colors.green,
    ),
    suffixText: 'km',
    border: OutlineInputBorder(
      borderRadius:
          BorderRadius.circular(12),
    ),
  ),
  validator: (value) {
    if (value == null ||
        value.trim().isEmpty) {
      return 'Please enter the distance';
    }

    final distance =
        double.tryParse(value.trim());

    if (distance == null) {
      return 'Enter a valid number';
    }

    if (!distance.isFinite ||
        distance <= 0) {
      return 'Distance must be greater than zero';
```

```
    }

    return null;
  },
  onChanged: (value) {
    setState(() {
      mileage = null;
    });
  },
),

const SizedBox(height: 22),

// FUEL INPUT
const Text(
  'Fuel Consumed (litres)',
  style: TextStyle(
    fontWeight: FontWeight.bold,
    fontSize: 15,
  ),
),

const SizedBox(height: 8),

TextFormField(
  controller: fuelController,
  keyboardType:
      const TextInputType.numberWithOptions(
    decimal: true,
  ),
  decoration: InputDecoration(
    hintText: 'Enter fuel consumed',
    prefixIcon: const Icon(
      Icons.local_gas_station,
      color: Colors.green,
    ),
    suffixText: 'L',
    border: OutlineInputBorder(
      borderRadius:
          BorderRadius.circular(12),
    ),
  ),
  validator: (value) {
    if (value == null ||
        value.trim().isEmpty) {
      return 'Please enter fuel consumed';
    }

    final fuel =
        double.tryParse(value.trim());

    if (fuel == null) {
      return 'Enter a valid number';
    }

    if (!fuel.isFinite || fuel <= 0) {
      return 'Fuel must be greater than zero';
    }

    return null;
  },
  onChanged: (value) {
    setState(() {
      mileage = null;
    });
  },
),
```

```
const SizedBox(height: 28),

// CALCULATE BUTTON
SizedBox(
  height: 50,
  child: ElevatedButton.icon(
    onPressed: calculateMileage,
    icon: const Icon(Icons.calculate),
    label: const Text(
      'Calculate Mileage',
      style: TextStyle(
        fontSize: 16,
        fontWeight: FontWeight.bold,
      ),
    ),
    style: ElevatedButton.styleFrom(
      backgroundColor: Colors.green,
      foregroundColor: Colors.white,
      shape: RoundedRectangleBorder(
        borderRadius:
            BorderRadius.circular(12),
      ),
    ),
  ),
),

const SizedBox(height: 12),

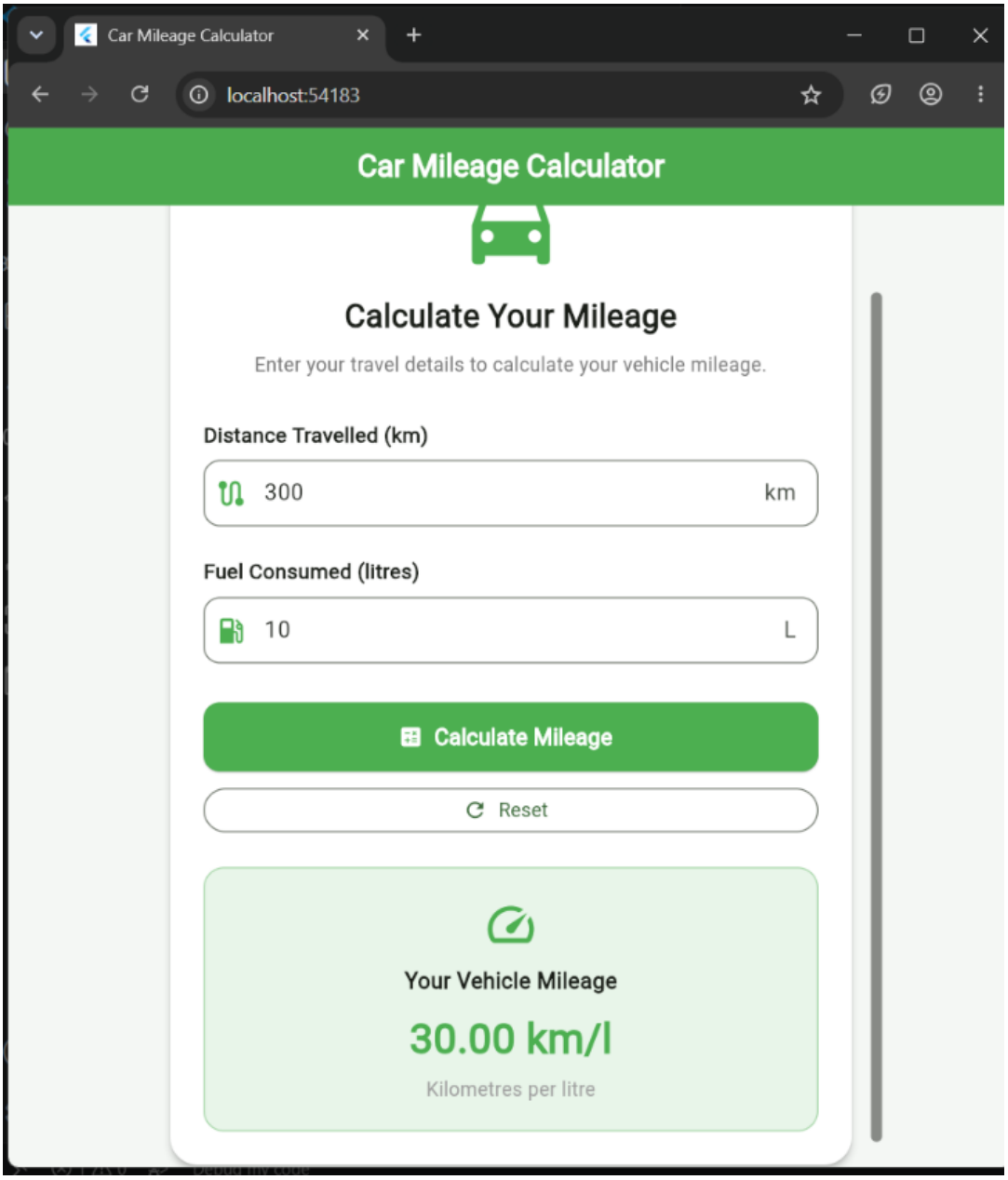
// RESET BUTTON
OutlinedButton.icon(
  onPressed: resetCalculator,
  icon: const Icon(Icons.refresh),
  label: const Text('Reset'),
),

// RESULT SECTION
if (mileage != null) ...[
  const SizedBox(height: 25),
  Container(
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      color: Colors.green.shade50,
      borderRadius:
          BorderRadius.circular(15),
      border: Border.all(
        color: Colors.green.shade200,
      ),
    ),
    child: Column(
      children: [
        const Icon(
          Icons.speed,
          size: 40,
          color: Colors.green,
        ),
        const SizedBox(height: 10),
        const Text(
          'Your Vehicle Mileage',
          style: TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 8),
        Text(
          '${mileage!.toStringAsFixed(2)} km/l',
          style: const TextStyle(
            fontSize: 30,
            color: Colors.green,
            fontWeight: FontWeight.bold,
```

```
                            ),
                          ),
                          const SizedBox(height: 5),
                          const Text(
                            'Kilometres per litre',
                            style: TextStyle(
                              color: Colors.grey,
                            ),
                          ),
                        ],
                      ),
                    ),
                  ],
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  );
  }
}
```

Output Screenshots

The screenshot below shows the Car Mileage Calculator running successfully in Google Chrome. The user has entered a distance of 300 km and fuel consumption of 10 litres. After selecting Calculate Mileage, the application validates the entries and displays the result as 30.00 km/l.



Result

Thus, the Car Mileage Calculator was successfully designed and implemented using Flutter Forms, TextFormField, input widgets and Stateful Widgets. The application validates the distance and fuel

inputs, calculates mileage using the distance-to-fuel formula and dynamically displays the result. For the tested values of 300 km and 10 litres, the calculated mileage was 30.00 km/l.

Ex. No. 5	Implementation of Navigation and Routing
Date of Exercise	31 October 2026
GitHub Code Link (Keep as public)	https://github.com/henrysam/FlutterLaboratory

24CS2016 – BUILDING APPLICATIONS USING FLUTTER

Aim

To understand and implement navigation and routing mechanisms in a Flutter application, enabling users to move between multiple screens and pass data between them using named routes.

Description

Navigation and routing allow users to move from one screen to another in a Flutter application. Flutter provides the Navigator class to manage the stack of routes. Named routes identify screens using route names and make navigation easier to organize.

This experiment implements a product navigation application named “ShopEase”. The application contains three screens: Home Screen, Product Details Screen and Shopping Cart Screen. The Home Screen displays a predefined list of products. Selecting a product opens its details screen, where the product name, description, image representation and price are displayed. The user can then navigate to the Cart Screen and view the selected product.

The main widgets and navigation concepts used are:

- MaterialApp: Configures the application, theme, initial route and named route table.
- routes: Maps route names such as '/', '/details' and '/cart' to their corresponding screen widgets.
- initialRoute: Specifies the first screen displayed when the application starts.
- Navigator.pushNamed(): Opens a new screen using its route name and can pass data through the arguments parameter.
- Navigator.pop(): Removes the current route and returns to the previous screen.
- ModalRoute.of(context): Retrieves the current route information and its arguments in the destination screen.
- Navigator.pushNamedAndRemoveUntil(): Navigates to a named route while removing previous routes that match the supplied condition.
- Product model: Holds the product name, description, emoji, price and background colour that are passed between screens.
- Reusable ProductCard: Displays a product in the Home Screen and handles the tap action.

The named routes are registered in MaterialApp. When a user selects a product on the Home Screen, Navigator.pushNamed() opens the '/details' route and passes the selected Product object through arguments. The Details Screen retrieves this object using ModalRoute.of(context)!.settings.arguments and displays its information. When the user selects Add to Cart, the application opens the '/cart' route and passes the same product object. The Cart Screen displays the selected product. The Go Back button uses Navigator.pop(), while Continue Shopping navigates to the Home Screen and removes the previous routes.

The application was executed in Google Chrome. The output screenshots demonstrate the Home Screen containing five products, the Product Details Screen showing the selected Wireless Headphones product, and the Shopping Cart Screen displaying the same product with a success message. The cart in this experiment is a navigation demonstration that displays the single product passed through route arguments; it does not maintain a persistent multi-product cart.

Program

File: lib/main.dart

```
import 'package:flutter/material.dart';


void main() {
  runApp(const ShopEaseApp());
}


// PRODUCT MODEL
class Product {
  final String name;
  final String description;
  final String emoji;
  final double price;
  final Color color;


  const Product({
    required this.name,
    required this.description,
    required this.emoji,
    required this.price,
    required this.color,
  });
}


// MAIN APPLICATION
class ShopEaseApp extends StatelessWidget {
  const ShopEaseApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'ShopEase Navigation',
      theme: ThemeData(
        colorSchemeSeed: Colors.deepPurple,
        useMaterial3: true,
      ),


      // NAMED ROUTES
      initialRoute: '/',
      routes: {
        '/': (context) => const HomeScreen(),
        '/details': (context) => const DetailsScreen(),
        '/cart': (context) => const CartScreen(),
      },
    );
  }
}


// HOME SCREEN
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});


  // PREDEFINED PRODUCTS
  static const List<Product> products = [
    Product(
      name: 'Wireless Headphones',
      description:
          'High-quality wireless headphones with clear sound '
          'and comfortable ear cushions.',
      emoji: '',
      price: 1499,
      color: Color(0xFFEDE7F6),
    ),
    Product(
```

```
    name: 'Smart Watch',
    description:
        'A stylish smartwatch with fitness tracking and '
        'smart notifications.',
    emoji: '⌚',
    price: 2499,
    color: Color(0xFFE3F2FD),
  ),
  Product(
    name: 'Running Shoes',
    description:
        'Comfortable and lightweight running shoes '
        'designed for everyday activities.',
    emoji: '👟',
    price: 1999,
    color: Color(0xFFFFF3E0),
  ),
  Product(
    name: 'Backpack',
    description:
        'A spacious and durable backpack suitable '
        'for college and travel.',
    emoji: '🎒',
    price: 999,
    color: Color(0xFFE8F5E9),
  ),
  Product(
    name: 'Camera',
    description:
        'Capture memorable moments with this '
        'high-quality digital camera.',
    emoji: '📷',
    price: 5999,
    color: Color(0xFFFCE4EC),
  ),
];

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F7FB),
    appBar: AppBar(
      backgroundColor: Colors.deepPurple,
      foregroundColor: Colors.white,
      title: const Text(
        'ShopEase',
        style: TextStyle(fontWeight: FontWeight.bold),
      ),
      actions: [
        IconButton(
          icon: const Icon(Icons.shopping_cart),
          onPressed: () {
            Navigator.pushNamed(context, '/cart');
          },
        ),
        const SizedBox(width: 10),
      ],
    ),

    body: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        // WELCOME SECTION
        Container(
          width: double.infinity,
          padding: const EdgeInsets.all(20),
          color: Colors.deepPurple.shade50,
          child: const Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'Welcome to ShopEase!',
                style: TextStyle(
```

```
                  fontSize: 22,
                  fontWeight: FontWeight.bold,
                  color: Colors.deepPurple,
                ),
              ),
              SizedBox(height: 5),
              Text('Explore our latest products.'),
            ],
          ),
        ),

        const Padding(
          padding: EdgeInsets.all(16),
          child: Text(
            'Available Products',
            style: TextStyle(
              fontSize: 20,
              fontWeight: FontWeight.bold,
            ),
          ),
        ),

        // PRODUCT LIST
        Expanded(
          child: ListView.builder(
            padding: const EdgeInsets.symmetric(horizontal: 14),
            itemCount: products.length,
            itemBuilder: (context, index) {
              final product = products[index];

              return ProductCard(
                product: product,
                onTap: () {
                  // NAVIGATE AND PASS DATA
                  Navigator.pushNamed(
                    context,
                    '/details',
                    arguments: product,
                  );
                },
              );
            },
          ),
        ),
      ],
    ),
  );
  }
}

// REUSABLE PRODUCT CARD
class ProductCard extends StatelessWidget {
  final Product product;
  final VoidCallback onTap;

  const ProductCard({
    super.key,
    required this.product,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.only(bottom: 14),
      elevation: 3,
      color: Colors.white,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(15),
      ),
```

```
      child: ListTile(
        contentPadding: const EdgeInsets.all(12),
        leading: Container(
          width: 65,
          height: 65,
          decoration: BoxDecoration(
            color: product.color,
            borderRadius: BorderRadius.circular(12),
          ),
          child: Center(
            child: Text(
              product.emoji,
              style: const TextStyle(fontSize: 35),
            ),
          ),
        ),
        title: Text(
          product.name,
          style: const TextStyle(fontWeight: FontWeight.bold),
        ),
        subtitle: Padding(
          padding: const EdgeInsets.only(top: 6),
          child: Text(
            '₹${product.price.toStringAsFixed(0)}',
            style: const TextStyle(
              color: Colors.deepPurple,
              fontWeight: FontWeight.bold,
            ),
          ),
        ),
        trailing: const Icon(
          Icons.arrow_forward_ios,
          size: 18,
        ),
        onTap: onTap,
      ),
    );
  }
}

// DETAILS SCREEN
class DetailsScreen extends StatelessWidget {
  const DetailsScreen({super.key});


  @override
  Widget build(BuildContext context) {
    // RECEIVE DATA FROM HOME SCREEN
    final product =
        ModalRoute.of(context)!.settings.arguments as Product;


    return Scaffold(
      backgroundColor: const Color(0xFFF7F7FB),
      appBar: AppBar(
        title: const Text('Product Details'),
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,
      ),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(20),
          child: Card(
            color: Colors.white,
            elevation: 4,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(20),
            ),
            child: Padding(
              padding: const EdgeInsets.all(24),
              child: Column(
                mainAxisSize: MainAxisSize.min,
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
```

```
// PRODUCT IMAGE
Container(
  height: 200,
  decoration: BoxDecoration(
    color: product.color,
    borderRadius: BorderRadius.circular(15),
  ),
  child: Center(
    child: Text(
      product.emoji,
      style: const TextStyle(fontSize: 100),
    ),
  ),
),

const SizedBox(height: 20),

// PRODUCT NAME
Text(
  product.name,
  textAlign: TextAlign.center,
  style: const TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
  ),
),

const SizedBox(height: 12),

// DESCRIPTION
Text(
  product.description,
  textAlign: TextAlign.center,
  style: const TextStyle(
    fontSize: 15,
    color: Colors.grey,
  ),
),

const SizedBox(height: 20),

// PRICE
Text(
  '₹${product.price.toStringAsFixed(0)}',
  textAlign: TextAlign.center,
  style: const TextStyle(
    fontSize: 28,
    color: Colors.deepPurple,
    fontWeight: FontWeight.bold,
  ),
),

const SizedBox(height: 25),

// ADD TO CART
ElevatedButton.icon(
  onPressed: () {
    Navigator.pushNamed(
      context,
      '/cart',
      arguments: product,
    );
  },
  icon: const Icon(Icons.shopping_cart),
  label: const Text('Add to Cart'),
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.deepPurple,
    foregroundColor: Colors.white,
    padding: const EdgeInsets.all(15),
```

```
                ),
              ),

              const SizedBox(height: 10),

              // BACK BUTTON
              OutlinedButton(
                onPressed: () {
                  Navigator.pop(context);
                },
                child: const Text('Go Back'),
              ),
            ],
          ),
        ),
      ),
    ),
  ),
);
  }
}

// CART SCREEN
class CartScreen extends StatelessWidget {
  const CartScreen({super.key});


  @override
  Widget build(BuildContext context) {
    // RECEIVE PRODUCT DATA
    final product =
        ModalRoute.of(context)!.settings.arguments as Product?;


    return Scaffold(
      backgroundColor: const Color(0xFFF7F7FB),
      appBar: AppBar(
        title: const Text('Shopping Cart'),
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,
      ),
      body: Center(
        child: product == null
            ? const Text(
                'Your cart is empty!',
                style: TextStyle(fontSize: 20),
              )
            : Padding(
                padding: const EdgeInsets.all(20),
                child: Card(
                  color: Colors.white,
                  child: Padding(
                    padding: const EdgeInsets.all(25),
                    child: Column(
                      mainAxisSize: MainAxisSize.min,
                      children: [
                        const Icon(
                          Icons.check_circle,
                          color: Colors.green,
                          size: 60,
                        ),

                        const SizedBox(height: 15),


                        const Text(
                          'Product Added Successfully!',
                          textAlign: TextAlign.center,
                          style: TextStyle(
                            fontSize: 20,
                            fontWeight: FontWeight.bold,
                          ),
```

```
                    ),

                    const SizedBox(height: 20),

                    Text(
                      product.emoji,
                      style: const TextStyle(fontSize: 65),
                    ),

                    const SizedBox(height: 10),

                    Text(
                      product.name,
                      style: const TextStyle(
                        fontSize: 20,
                        fontWeight: FontWeight.bold,
                      ),
                    ),

                    const SizedBox(height: 8),

                    Text(
                      '₹${product.price.toStringAsFixed(0)}',
                      style: const TextStyle(
                        fontSize: 22,
                        color: Colors.deepPurple,
                        fontWeight: FontWeight.bold,
                      ),
                    ),

                    const SizedBox(height: 25),

                    ElevatedButton(
                      onPressed: () {
                        Navigator.pushNamedAndRemoveUntil(
                          context,
                          '/',
                          (route) => false,
                        );
                      },
                      style: ElevatedButton.styleFrom(
                        backgroundColor: Colors.deepPurple,
                        foregroundColor: Colors.white,
                      ),
                      child: const Text('Continue Shopping'),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ),
      );
  }
}
```

Output Screenshots

The following screenshots show the three screens of the ShopEase Navigation application running in Google Chrome. Figure 1 shows the Home Screen with the available products. Figure 2 shows the Product Details Screen displaying the selected Wireless Headphones product and its details. Figure 3 shows the Shopping Cart Screen displaying the same product after the Add to Cart action.

Figure 1: Home Screen displaying available products

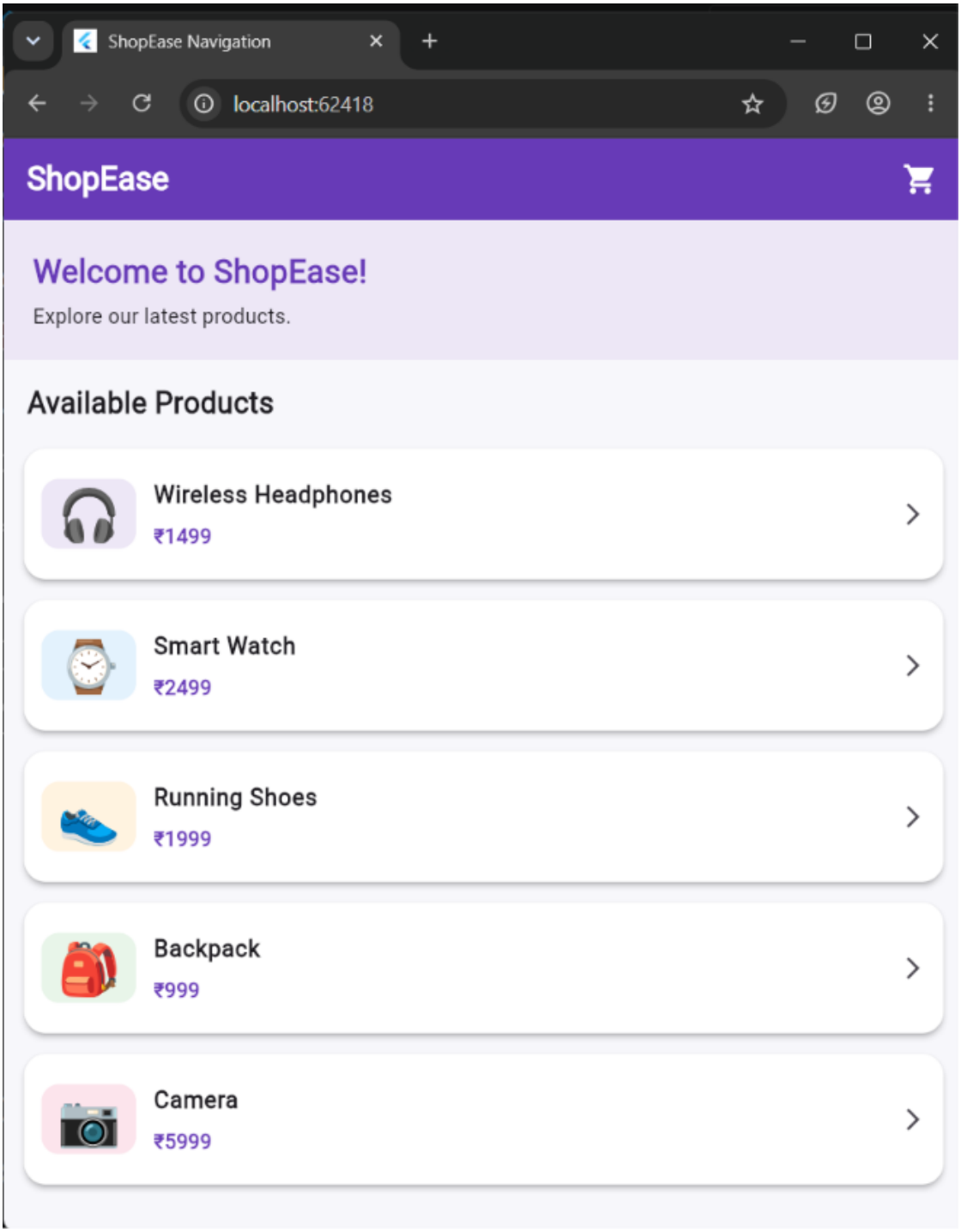


Figure 2: Product Details Screen showing the selected product

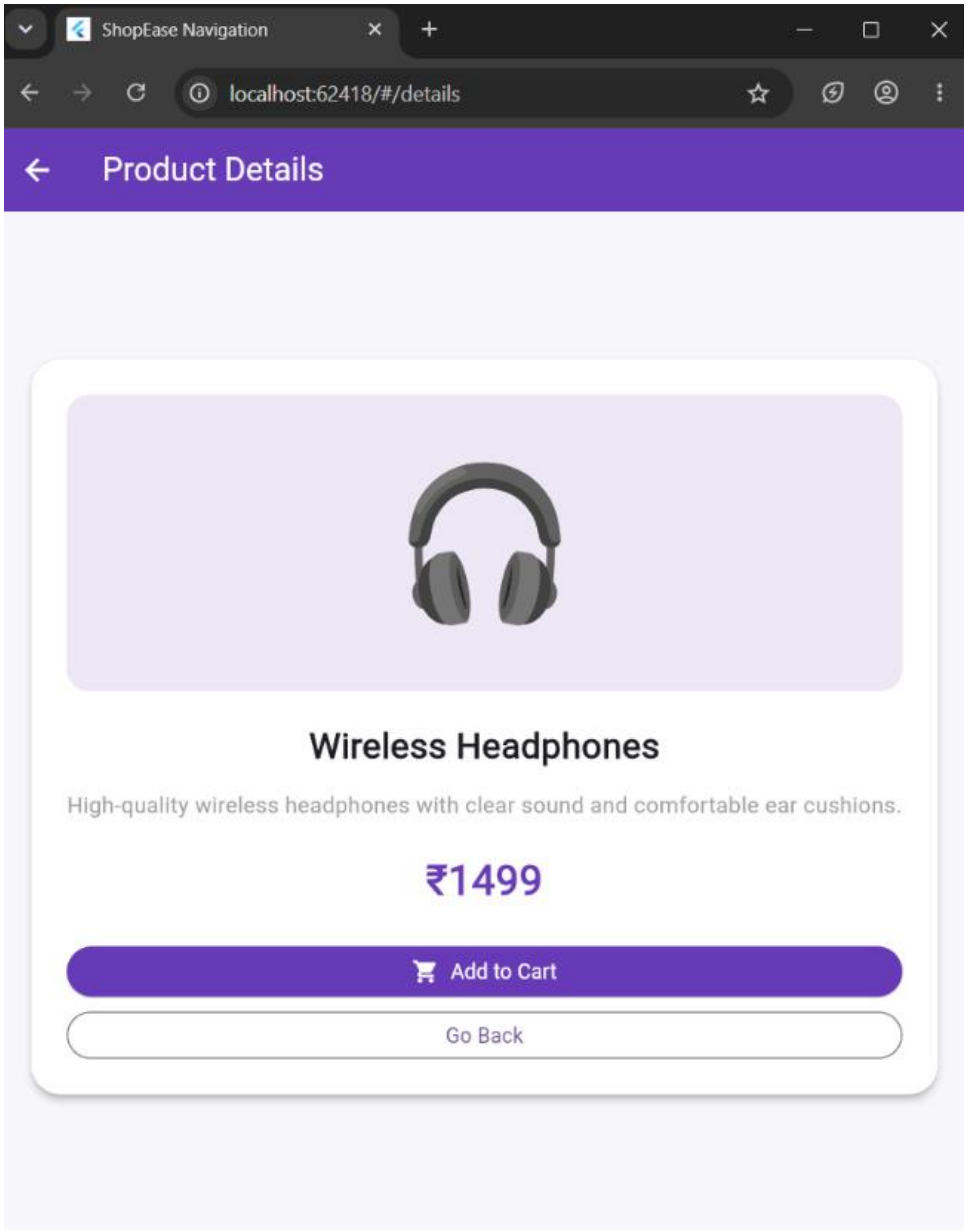
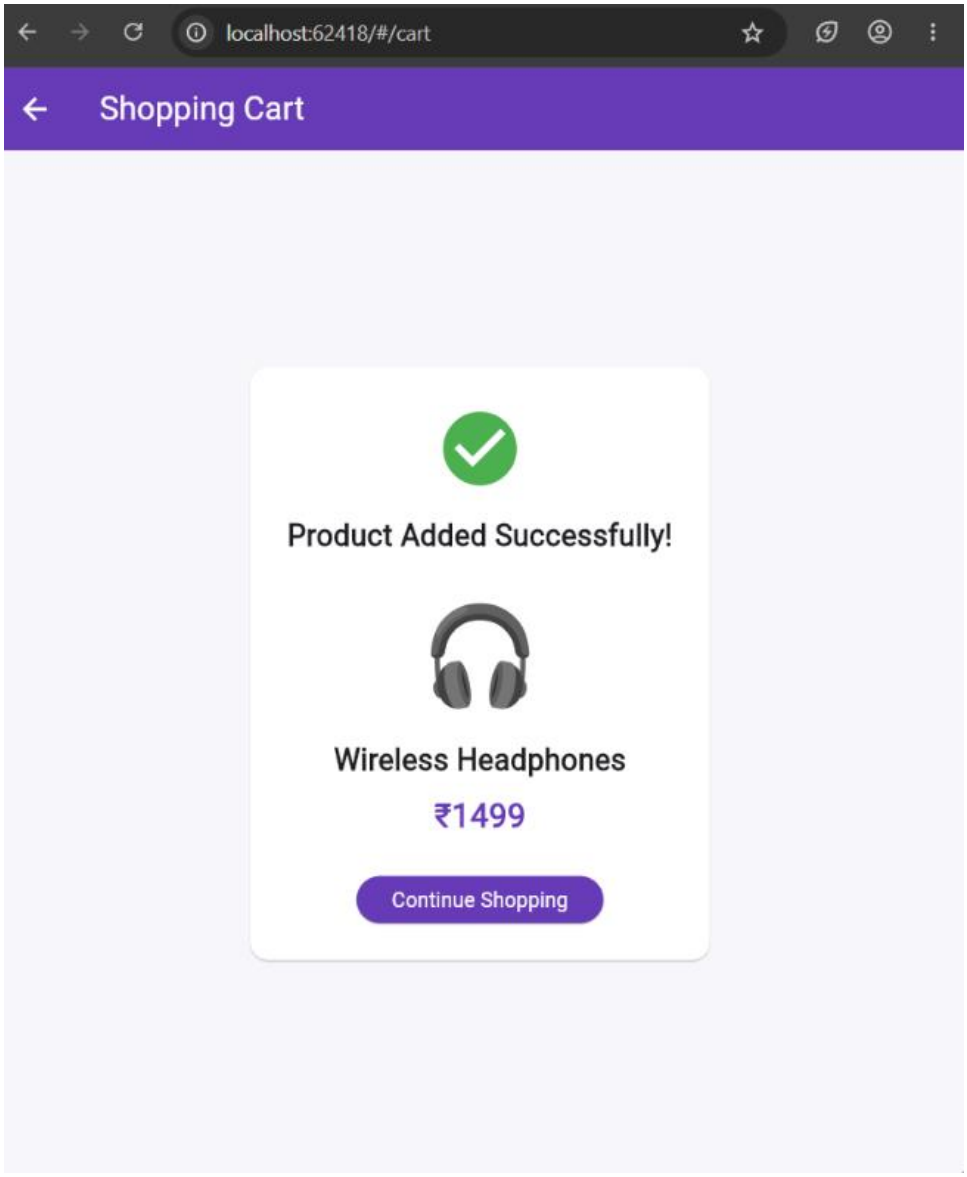


Figure 3: Shopping Cart Screen showing the added product



Result

Thus, navigation and routing were successfully implemented in the Flutter ShopEase application using named routes. The user can navigate from the Home Screen to the Product Details Screen and then to the Shopping Cart Screen. Product information was passed between screens using route arguments and retrieved using ModalRoute.of(context). The application was executed successfully in Google Chrome.

Ex. No. 6	Data Persistence Using SQLite
Date of Exercise	01 November 2026
Github Code Link(Keep as public)	https://github.com/henrysam/FlutterLaboratory

Aim

To design and develop a Flutter application that stores and retrieves student records using an SQLite database by implementing Create and Read operations.

Description

Data persistence is the process of storing application data so that it remains available even after the application is closed or refreshed. SQLite is a lightweight relational database that stores information in tables. The sqflite package provides SQLite database operations in Flutter. Since this experiment is executed in Google Chrome, the sqflite_common_ffi_web package is used to enable SQLite support on the web platform.

The application is named StudentHub. It allows the user to enter a student's name, register number, department and email address. When the Save Student button is pressed, the application validates the entered details and inserts the record into the students table. The application then retrieves the saved records and displays them in the Student Records section.

The students table contains the following fields:
• id – INTEGER PRIMARY KEY AUTOINCREMENT
• name – TEXT NOT NULL
• registerNumber – TEXT NOT NULL
• department – TEXT NOT NULL
• email – TEXT NOT NULL

The main concepts used are DatabaseHelper for database initialization and operations, CREATE TABLE for creating the students table, INSERT for storing student details, SELECT for retrieving records, the Student model for converting data between objects and maps, and StatefulWidget with setState() for updating the interface.

The application was executed in Google Chrome at localhost:5555. The output screenshots show the student registration form and a saved student record displayed in the Student Records list.

Program

```
import 'package:flutter/material.dart';
import 'package:flutter/foundation.dart';
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';
import 'package:path/path.dart' as p;

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  if (kIsWeb) {
    databaseFactory = databaseFactoryFfiWeb;
  }
  runApp(const StudentHubApp());
}

// STUDENT MODEL
class Student {
  final int? id;
  final String name, registerNumber, department, email;
```

```
const Student({
  this.id,
  required this.name,
  required this.registerNumber,
  required this.department,
  required this.email,
});

Map<String, dynamic> toMap() => {
  'name': name,
  'registerNumber': registerNumber,
  'department': department,
  'email': email,
};

factory Student.fromMap(Map<String, dynamic> map) => Student(
  id: map['id'] as int,
  name: map['name'] as String,
  registerNumber: map['registerNumber'] as String,
  department: map['department'] as String,
  email: map['email'] as String,
);
```

```
}

// DATABASE HELPER
class DatabaseHelper {
  DatabaseHelper._();
  static final DatabaseHelper instance = DatabaseHelper._();
  static Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    final String databasePath;
    if (kIsWeb) {
      databasePath = 'studenthub.db';
    } else {
      final path = await getDatabasesPath();
      databasePath = p.join(path, 'studenthub.db');
    }
```

```
  return openDatabase(
    databasePath,
    version: 1,
    onCreate: (db, version) async {
      await db.execute('''
        CREATE TABLE students (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          name TEXT NOT NULL,
          registerNumber TEXT NOT NULL,
          department TEXT NOT NULL,
          email TEXT NOT NULL
        )
      ''');
    },
  );
}

// CREATE operation
Future<int> insertStudent(Student student) async {
  final db = await database;
  return db.insert('students', student.toMap(),
      conflictAlgorithm: ConflictAlgorithm.abort);
}
```

```
  // READ operation
  Future<List<Student>> getStudents() async {
    final db = await database;
    final result = await db.query('students', orderBy: 'id DESC');
    return result.map((map) => Student.fromMap(map)).toList();
  }
}

// MAIN APPLICATION
class StudentHubApp extends StatelessWidget {
  const StudentHubApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'StudentHub',
      theme: ThemeData(
        colorSchemeSeed: const Color(0xFF4053B5),
        useMaterial3: true,
      ),
      home: const StudentHomePage(),
```

```
    );
  }
}

// STUDENT HOME PAGE
class StudentHomePage extends StatefulWidget {
  const StudentHomePage({super.key});

  @override
  State<StudentHomePage> createState() => _StudentHomePageState();
}

class _StudentHomePageState extends State<StudentHomePage> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _registerController = TextEditingController();
  final _departmentController = TextEditingController();
  final _emailController = TextEditingController();

  List<Student> students = [];
  bool isLoading = true;
  bool isSaving = false;
```

```
@override
void initState() {
  super.initState();
  loadStudents();
}

// READ saved records
Future<void> loadStudents() async {
  final records = await DatabaseHelper.instance.getStudents();
  if (!mounted) return;
  setState(() {
    students = records;
    isLoading = false;
  });
}

// CREATE a new record
Future<void> saveStudent() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() => isSaving = true);

  final student = Student(
    name: _nameController.text.trim(),
```

```
    registerNumber: _registerController.text.trim(),
    department: _departmentController.text.trim(),
    email: _emailController.text.trim(),
  );

  try {
    await DatabaseHelper.instance.insertStudent(student);
    _nameController.clear();
    _registerController.clear();
    _departmentController.clear();
    _emailController.clear();
    await loadStudents();

    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Student saved successfully!')),
    );
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Unable to save student: $e')),
    );
  } finally {
```

```
    if (mounted) setState(() => isSaving = false);
  }
}

String? requiredValidator(String? value) {
  if (value == null || value.trim().isEmpty) {
    return 'This field is required';
  }
  return null;
}

@override
void dispose() {
  _nameController.dispose();
  _registerController.dispose();
  _departmentController.dispose();
  _emailController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
```

```
backgroundColor: const Color(0xFFF4F5FA),
appBar: AppBar(
  title: const Text('StudentHub',
      style: TextStyle(fontWeight: FontWeight.bold)),
  backgroundColor: const Color(0xFF4053B5),
  foregroundColor: Colors.white,
  actions: [
    IconButton(
      icon: const Icon(Icons.refresh),
      onPressed: loadStudents,
    ),
  ],
),
body: Center(
  child: ConstrainedBox(
    constraints: const BoxConstraints(maxWidth: 750),
    child: ListView(
      padding: const EdgeInsets.all(16),
      children: [
        Container(
          padding: const EdgeInsets.all(20),
          decoration: BoxDecoration(
            color: const Color(0xFFE8EAF6),
```

```
    borderRadius: BorderRadius.circular(16),
  ),
  child: const Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text('Student Record Management',
          style: TextStyle(fontSize: 21,
              fontWeight: FontWeight.bold,
              color: Color(0xFF4053B5))),
      SizedBox(height: 6),
      Text('Add and manage student records using SQLite database.'),
    ],
  ),
),
const SizedBox(height: 22),
Card(
  color: Colors.white,
  elevation: 3,
  child: Padding(
    padding: const EdgeInsets.all(18),
    child: Form(
      key: _formKey,
      child: Column(
```

```
crossAxisAlignment: CrossAxisAlignment.start,
children: [
 const Text('Add New Student',
    style: TextStyle(fontSize: 20,
       fontWeight: FontWeight.bold)),
 const SizedBox(height: 18),
 _buildField('Student Name', 'Enter student name',
    Icons.person, _nameController),
 _buildField('Register Number', 'Enter register number',
    Icons.badge, _registerController),
 _buildField('Department', 'Enter department',
    Icons.school, _departmentController),
 _buildField('Email Address', 'Enter email address',
    Icons.email, _emailController, email: true),
 const SizedBox(height: 8),
 SizedBox(
  width: double.infinity,
  height: 48,
  child: ElevatedButton.icon(
   onPressed: isSaving ? null : saveStudent,
   icon: const Icon(Icons.save),
   label: Text(isSaving ? 'Saving...' : 'Save Student'),
   style: ElevatedButton.styleFrom(
```

```
                backgroundColor: const Color(0xFF4053B5),
                foregroundColor: Colors.white,
              ),
            ),
          ),
        ],
      ),
    ),
  ),
),
const SizedBox(height: 24),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    const Text('Student Records',
        style: TextStyle(fontSize: 20,
            fontWeight: FontWeight.bold)),
    Chip(label: Text('${students.length} Students')),
  ],
),
const SizedBox(height: 8),
if (isLoading)
  const Center(child: CircularProgressIndicator())
```

```
          else if (students.isEmpty)
            const Card(
              child: Padding(
                padding: EdgeInsets.all(24),
                child: Center(child: Text('No student records found.')),
              ),
            )
          else
            ...students.map((student) => StudentCard(student: student)),
        ],
      ),
    ),
  ),
 );
}

Widget _buildField(String label, String hint, IconData icon,
    TextEditingController controller, {bool email = false}) {
  return Padding(
    padding: const EdgeInsets.only(bottom: 14),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
```

```
Text(label, style: const TextStyle(fontWeight: FontWeight.w600)),
const SizedBox(height: 7),
TextFormField(
  controller: controller,
  validator: (value) {
    final error = requiredValidator(value);
    if (error != null) return error;
    if (email &&
        !RegExp(r'^[^@]+@[^@]+\.[^@]+$').hasMatch(value!.trim())) {
      return 'Enter a valid email address';
    }
    return null;
  },
  decoration: InputDecoration(
    hintText: hint,
    prefixIcon: Icon(icon, color: const Color(0xFF4053B5)),
    filled: true,
    fillColor: const Color(0xFFF7F7FB),
    border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(11)),
  ),
 ),
],
```

```
      ),
    );
  }
}

// REUSABLE STUDENT CARD
class StudentCard extends StatelessWidget {
  final Student student;
  const StudentCard({super.key, required this.student});

  @override
  Widget build(BuildContext context) {
    return Card(
      color: Colors.white,
      elevation: 2,
      margin: const EdgeInsets.only(bottom: 12),
      child: ListTile(
        contentPadding: const EdgeInsets.all(14),
        leading: CircleAvatar(
          backgroundColor: const Color(0xFFC5CAE9),
          child: Text(student.name.isNotEmpty
              ? student.name[0].toUpperCase() : '?',
              style: const TextStyle(color: Color(0xFF4053B5),
```

```
            fontWeight: FontWeight.bold)),
      ),
      title: Text(student.name,
        style: const TextStyle(fontWeight: FontWeight.bold)),
      subtitle: Padding(
       padding: const EdgeInsets.only(top: 6),
       child: Text(
        'Register No: ${student.registerNumber}\n'
        'Department: ${student.department}\n'
        'Email: ${student.email}',
       ),
      ),
      trailing: const Icon(Icons.storage, color: Colors.green),
    ),
   );
  }
}
```

Output Screenshots

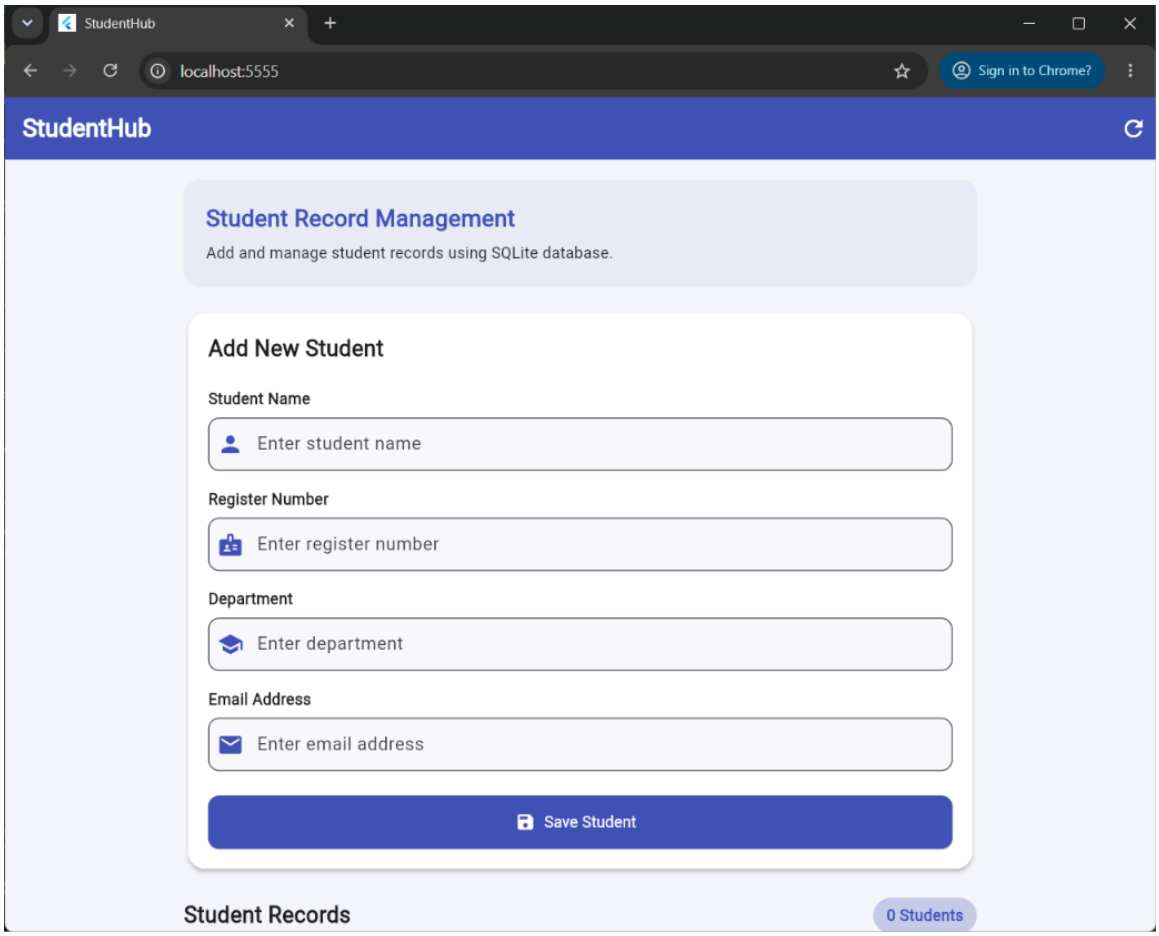


Figure 1: StudentHub application displaying the student registration form.

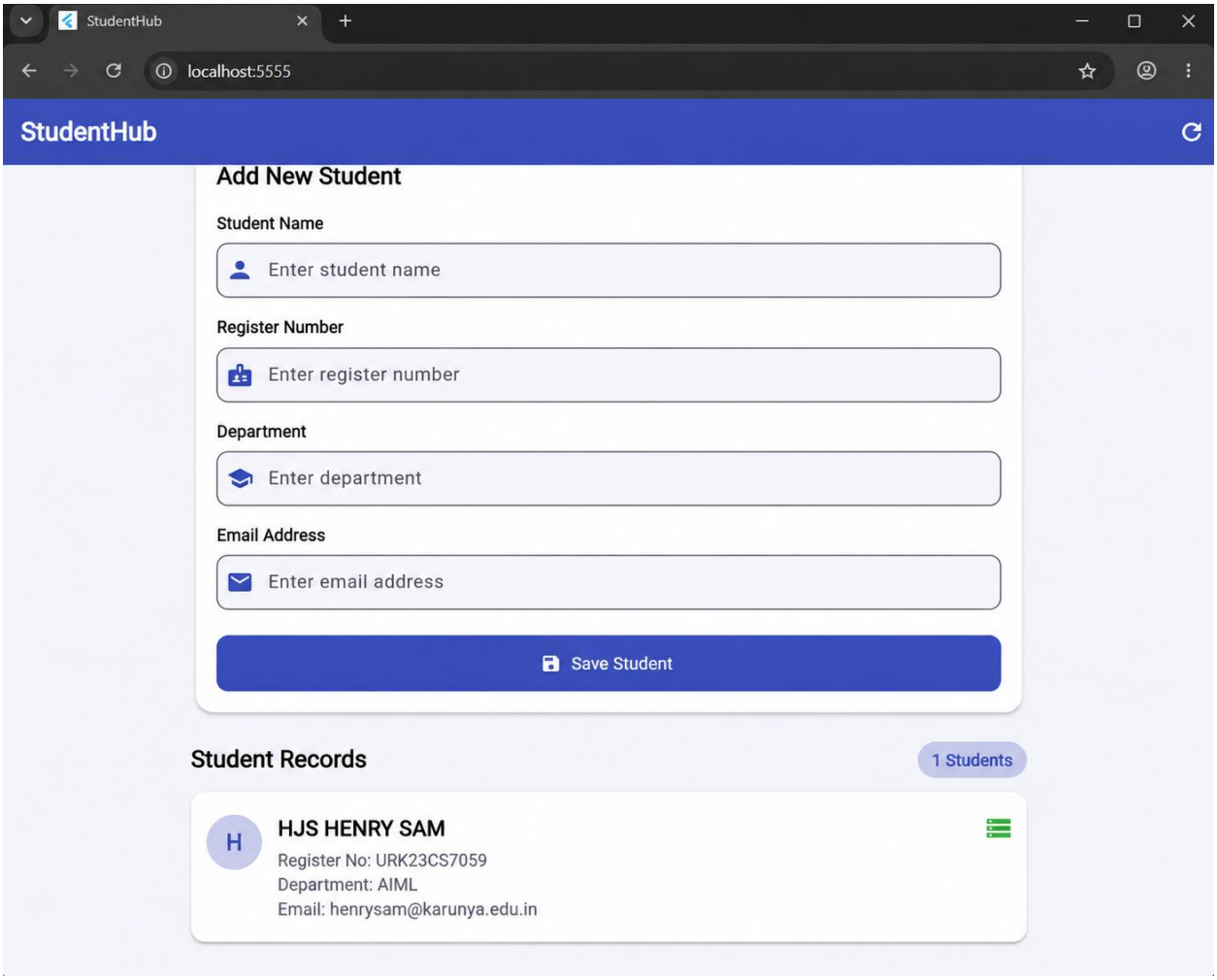


Figure 2: Student record successfully saved and displayed in the Student Records list.

Result

Thus, the Flutter StudentHub application was successfully developed to perform Create and Read operations using an SQLite database. Student details entered through the form were inserted into the students table and retrieved from the database for display in the application. The application was executed successfully in Google Chrome using SQLite Web support.

Ex. No. 7	Data Persistence Using Google Cloud Firestore
Date of Exercise	01 November 2026
Github Code Link(Keep as public)	https://github.com/henrysam/FlutterLaboratory

Aim

To design and develop a Flutter application that stores and retrieves student information using Google Cloud Firestore, demonstrating cloud-based data persistence through Create and Read operations.

Description

Cloud Firestore is a NoSQL cloud database provided by Firebase. It stores data as documents organized into collections and synchronizes changes in real time. In this experiment, a Flutter application named StudentHub Cloud is developed to register student details and retrieve the stored records from a Firestore collection named 'students'. The application collects the student's name, register number, department and email address. When the Save Student button is pressed, the add() method of the Cloud Firestore collection reference creates a new document containing these fields and a server timestamp. A StreamBuilder listens to the collection's snapshots() stream and displays the student records in the application. This stream-based approach updates the displayed list when Firestore data changes. Firebase is initialized in the main() function using Firebase.initializeApp() and the platform-specific configuration generated by the FlutterFire CLI. The application was executed in Google Chrome, and the saved student document was verified in the Firebase Console.

Program

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  runApp(const MyApp());
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'StudentHub Cloud',
      theme: ThemeData(
        primarySwatch: Colors.indigo,
        useMaterial3: true,
      ),
      home: const StudentPage(),
    );
  }
}

class StudentPage extends StatefulWidget {
  const StudentPage({super.key});

  @override
  State<StudentPage> createState() => _StudentPageState();
}

class _StudentPageState extends State<StudentPage> {
  final nameController = TextEditingController();
  final registerController = TextEditingController();
  final departmentController = TextEditingController();
  final emailController = TextEditingController();

  final CollectionReference students =
      FirebaseFirestore.instance.collection('students');

  Future<void> addStudent() async {
    if (nameController.text.isEmpty ||
        registerController.text.isEmpty ||
        departmentController.text.isEmpty ||
        emailController.text.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Please fill all fields')),
      );
      return;
    }

    await students.add({
      'name': nameController.text.trim(),
      'registerNumber': registerController.text.trim(),
      'department': departmentController.text.trim(),
      'email': emailController.text.trim(),
      'createdAt': FieldValue.serverTimestamp(),
    });

    nameController.clear();
    registerController.clear();
```

```
  departmentController.clear();
  emailController.clear();

  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Student saved successfully!')),
  );
}

@override
void dispose() {
  nameController.dispose();
  registerController.dispose();
  departmentController.dispose();
  emailController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('StudentHub Cloud'),
      backgroundColor: Colors.indigo,
      foregroundColor: Colors.white,
    ),
    body: Padding(
      padding: const EdgeInsets.all(20),
      child: Column(
        children: [
          TextField(
            controller: nameController,
            decoration: const InputDecoration(
              labelText: 'Student Name',
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 12),

          TextField(
            controller: registerController,
            decoration: const InputDecoration(
              labelText: 'Register Number',
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 12),

          TextField(
            controller: departmentController,
            decoration: const InputDecoration(
              labelText: 'Department',
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 12),
```

```
TextField(
  controller: emailController,
  decoration: const InputDecoration(
    labelText: 'Email',
    border: OutlineInputBorder(),
  ),
),
const SizedBox(height: 15),

ElevatedButton.icon(
  onPressed: addStudent,
  icon: const Icon(Icons.cloud_upload),
  label: const Text('Save Student'),
),

const Divider(height: 30),

const Text(
  'Registered Students',
  style: TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.bold,
  ),
),

Expanded(
  child: StreamBuilder<QuerySnapshot>(
    stream: students
        .orderBy('createdAt', descending: true)
        .snapshots(),
    builder: (context, snapshot) {
      if (snapshot.hasError) {
        return Text('Error: ${snapshot.error}');
      }

      if (snapshot.connectionState ==
          ConnectionState.waiting) {
        return const CircularProgressIndicator();
      }

      final studentList = snapshot.data!.docs;

      if (studentList.isEmpty) {
        return const Center(
          child: Text('No students registered yet'),
        );
      }

      return ListView.builder(
        itemCount: studentList.length,
        itemBuilder: (context, index) {
          final data = studentList[index].data()
              as Map<String, dynamic>;

          return Card(
```

```
                child: ListTile(
                  leading: const CircleAvatar(
                    child: Icon(Icons.person),
                  ),
                  title: Text(data['name']),
                  subtitle: Text(
                    'Register No: ${data['registerNumber']}\n'
                    'Department: ${data['department']}\n'
                    'Email: ${data['email']}',
                  ),
                  isThreeLine: true,
                ),
              );
            },
          );
        },
      ),
    ),
  ],
 ),
 ),
 );
 }
}
```

Output Screenshots

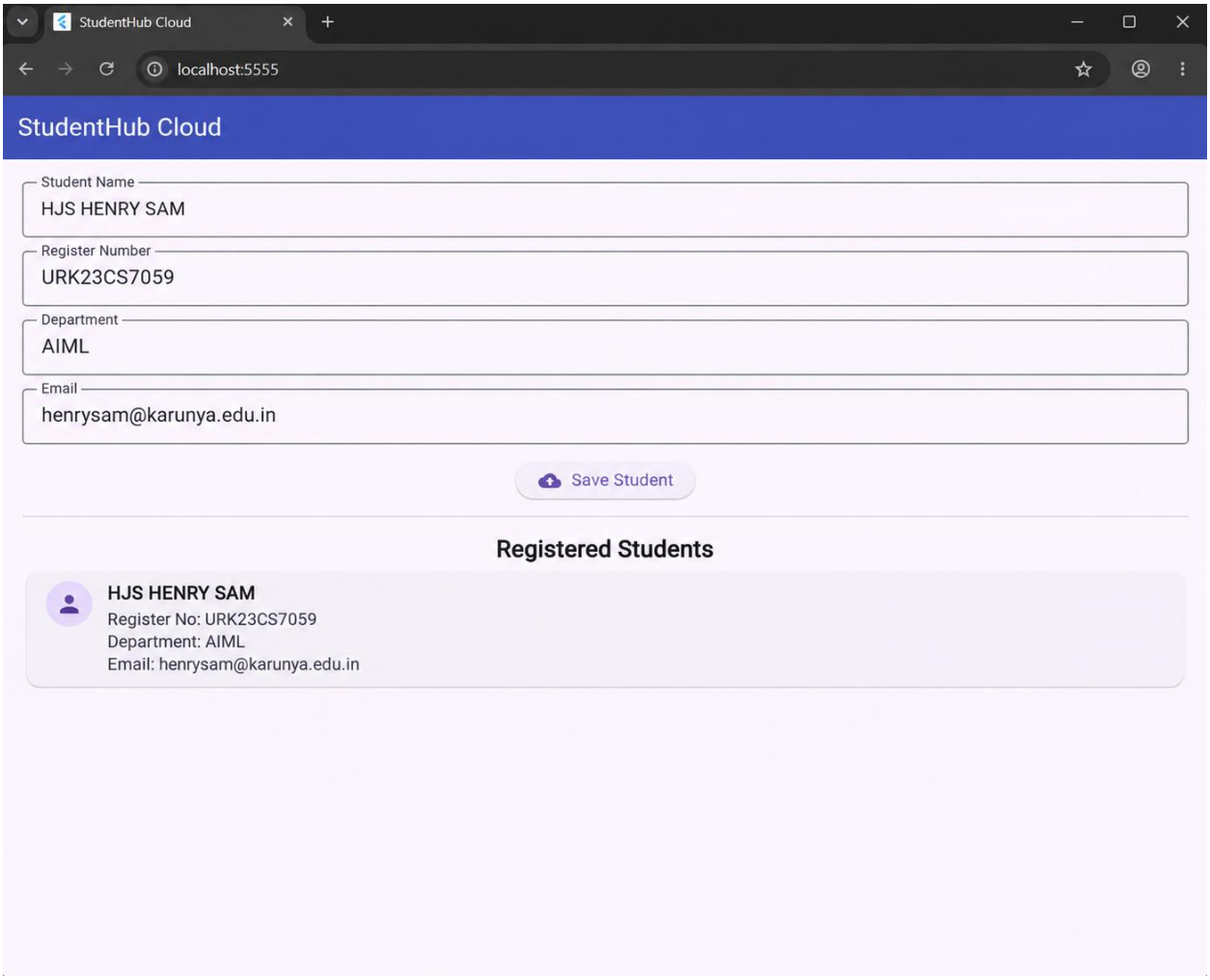


Figure 1: StudentHub Cloud application displaying the student registration form and retrieved student record.

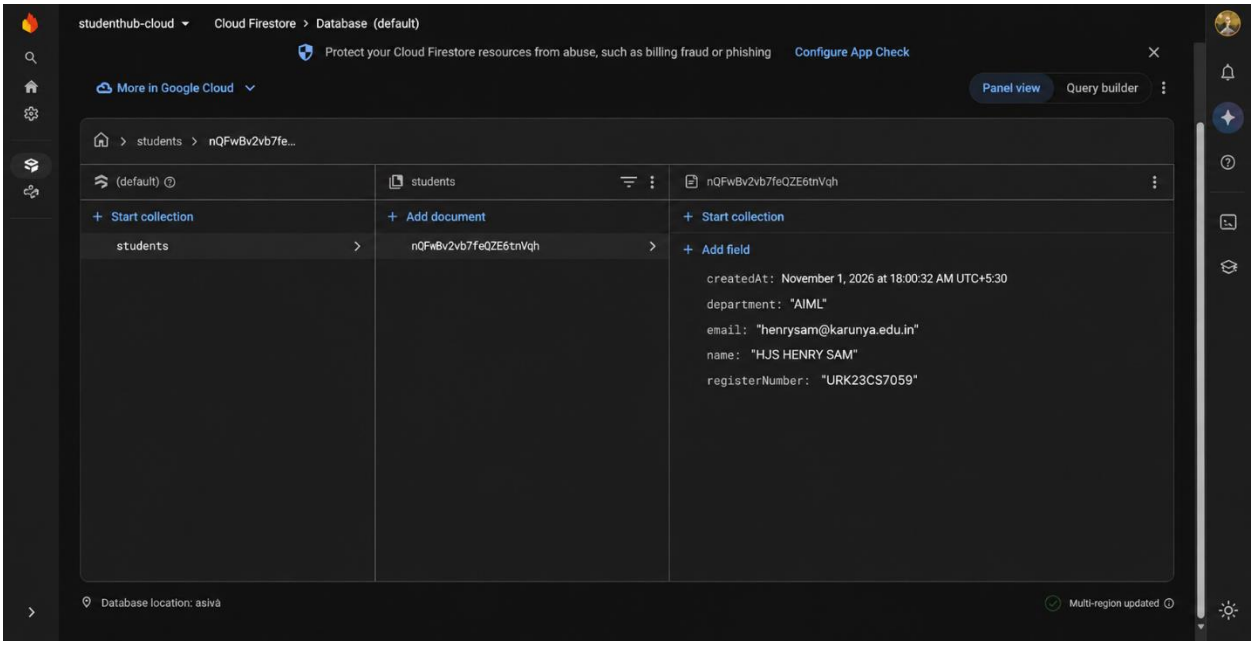


Figure 2: Student record stored successfully in the 'students' collection of Google Cloud Firestore.

// Output of the program

Result

Thus, the StudentHub Cloud Flutter application was successfully developed and connected to Google Cloud Firestore. The application performed Create and Read operations by storing student details in the 'students' collection and retrieving them using a real-time snapshots() stream. The saved student document was verified in the Firebase Console.

Ex. No. 8	Searching Information in Google Firestore Database
Date of Exercise	01 November 2026
Github Code Link(Keep as public)	https://github.com/henrysam/FlutterLaboratory

Aim

To develop a Flutter application that retrieves information from Google Cloud Firestore based on user search input and displays the relevant student information on the screen.

Description

Google Cloud Firestore is a cloud-hosted NoSQL database that stores data in collections and documents. In this experiment, a Flutter application named Student Search is developed to search for student information stored in the 'students' collection created in Experiment 7. The application provides a text field in which the user enters a student's name. When the Search button is pressed or the Enter key is submitted, the application performs a Firestore query using the where() method with isGreaterThanOrEqualTo and isLessThanOrEqualTo conditions. The query retrieves names beginning with the entered search text. The get() method fetches the matching documents from Firestore. The results are stored in a list and displayed using ListView.builder and ListTile widgets. If no matching document is found, the application displays an appropriate message. A clear button resets the search field and removes the displayed results. Firebase is initialized using Firebase.initializeApp() and the configuration generated by the FlutterFire CLI. The application was executed in Google Chrome and successfully retrieved the student record for the search term 'Santhosh'.

Program

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
```

```
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Student Search',
      theme: ThemeData(
        primarySwatch: Colors.indigo,
        useMaterial3: true,
      ),
      home: const SearchPage(),
    );
  }
}

class SearchPage extends StatefulWidget {
  const SearchPage({super.key});

  @override
  State<SearchPage> createState() => _SearchPageState();
}

class _SearchPageState extends State<SearchPage> {
  final TextEditingController searchController =
      TextEditingController();

  List<QueryDocumentSnapshot> results = [];
  bool isLoading = false;
  bool hasSearched = false;

  Future<void> searchStudents(String value) async {
    if (value.trim().isEmpty) {
      setState(() {
        results = [];
        hasSearched = false;
      });
      return;
    }

    setState(() {
      isLoading = true;
      hasSearched = true;
    });

    try {
      final query = await FirebaseFirestore.instance
          .collection('students')
          .where('name',
              isGreaterThanOrEqualTo: value.trim())
          .where('name',
```

```
          isLessThanOrEqualTo: '${value.trim()}\uf8ff')
      .get();

   setState(() {
    results = query.docs;
    isLoading = false;
   });
  } catch (e) {
   setState(() {
    isLoading = false;
   });

   if (!mounted) return;

   ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Search failed: $e')),
   );
  }
}

@override
void dispose() {
 searchController.dispose();
 super.dispose();
}

@override
Widget build(BuildContext context) {
 return Scaffold(
   appBar: AppBar(
    title: const Text('Student Search'),
    backgroundColor: Colors.indigo,
    foregroundColor: Colors.white,
   ),
   body: Padding(
    padding: const EdgeInsets.all(20),
    child: Column(
     children: [
      TextField(
       controller: searchController,
       decoration: InputDecoration(
        labelText: 'Search student by name',
        hintText: 'Enter student name',
        prefixIcon: const Icon(Icons.search),
        suffixIcon: IconButton(
         icon: const Icon(Icons.clear),
         onPressed: () {
          searchController.clear();
          searchStudents('');
         },
        ),
        border: const OutlineInputBorder(),
       ),
       onSubmitted: searchStudents,
      ),
```

```
const SizedBox(height: 12),

ElevatedButton.icon(
  onPressed: () =>
      searchStudents(searchController.text),
  icon: const Icon(Icons.search),
  label: const Text('Search'),
),

const SizedBox(height: 20),

const Text(
  'Search Results',
  style: TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.bold,
  ),
),

const SizedBox(height: 10),

Expanded(
  child: isLoading
      ? const Center(
          child: CircularProgressIndicator(),
        )
      : !hasSearched
          ? const Center(
              child: Text(
                'Enter a student name to search',
              ),
            )
          : results.isEmpty
              ? const Center(
                  child: Text(
                    'No matching students found',
                  ),
                )
              : ListView.builder(
                  itemCount: results.length,
                  itemBuilder: (context, index) {
                    final data = results[index].data()
                        as Map<String, dynamic>;

                    return Card(
                      child: ListTile(
                        leading: const CircleAvatar(
                          child: Icon(Icons.person),
                        ),
                        title: Text(data['name']),
                        subtitle: Text(
                          'Register No: ${data['registerNumber']}\n'
                          'Department: ${data['department']}\n'
                          'Email: ${data['email']}',
                        ),
                        isThreeLine: true,
```

```
                    ),
                  );
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output Screenshots

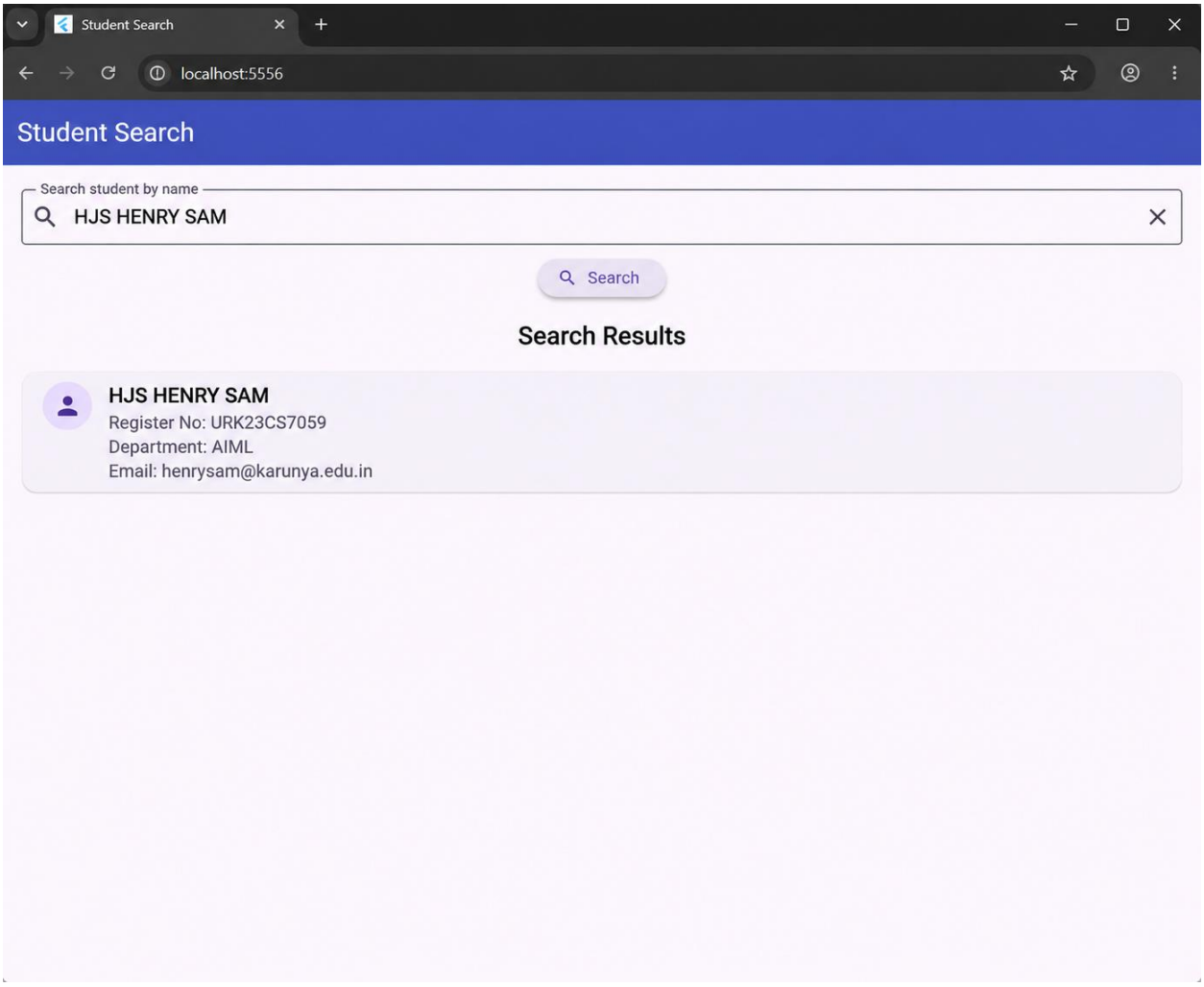


Figure 1: Student Search application displaying the student record retrieved from Google Cloud Firestore for the search term 'Santhosh'.

// Output of the program

Result

Thus, the Flutter application for searching information in Google Cloud Firestore was successfully developed and executed. The application accepted a search term, queried the 'students' collection, retrieved the matching student document and displayed the student's name, register number, department and email address on the screen.

Ex. No. 9	Update Information in Google Firestore Database
Date of Exercise	01 November 2026
Github Code Link(Keep as public)	https://github.com/henrysam/FlutterLaboratory

Aim

To develop a Flutter application that updates existing student information in Google Cloud Firestore based on user input and displays the updated information on the screen.

Description

Google Cloud Firestore is a cloud-hosted NoSQL database that stores information in collections and documents. In this experiment, a Flutter application named Student Update is developed to modify student information already stored in the 'students' collection. The user enters a student's register number and selects the Find Student button. The application searches Firestore using the where() method with an equality condition on the registerNumber field. If a matching document is found, its document ID and current student details are retrieved and displayed in editable text fields. The user can modify the student's name, department and email address. When the Update Student button is pressed, the application uses the Firestore document reference and update() method to modify the existing document. The register number is used to identify the record and is not changed. A success message is displayed after the update operation completes. Firebase is initialized using Firebase.initializeApp() and the configuration generated by the FlutterFire CLI. The application was run in Google Chrome, and the successful update operation was verified by the displayed confirmation message.

Program

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
```

```
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Student Update',
      theme: ThemeData(
        primarySwatch: Colors.indigo,
        useMaterial3: true,
      ),
      home: const UpdatePage(),
    );
  }
}

class UpdatePage extends StatefulWidget {
  const UpdatePage({super.key});

  @override
  State<UpdatePage> createState() => _UpdatePageState();
}

class _UpdatePageState extends State<UpdatePage> {
  final registerController = TextEditingController();
  final nameController = TextEditingController();
  final departmentController = TextEditingController();
  final emailController = TextEditingController();

  final CollectionReference students =
      FirebaseFirestore.instance.collection('students');

  String? documentId;
  bool isLoading = false;

  Future<void> searchStudent() async {
    final registerNumber = registerController.text.trim();

    if (registerNumber.isEmpty) return;

    setState(() {
      isLoading = true;
      documentId = null;
    });

    try {
      final query = await students
          .where('registerNumber', isEqualTo: registerNumber)
          .get();
```

```
    if (query.docs.isNotEmpty) {
      final doc = query.docs.first;
      final data = doc.data() as Map<String, dynamic>;

      setState(() {
        documentId = doc.id;
        nameController.text = data['name'] ?? '';
        departmentController.text = data['department'] ?? '';
        emailController.text = data['email'] ?? '';
        isLoading = false;
      });
    } else {
      setState(() {
        nameController.clear();
        departmentController.clear();
        emailController.clear();
        isLoading = false;
      });

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Student not found')),
      );
    }
  } catch (e) {
    setState(() => isLoading = false);

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Search failed: $e')),
    );
  }
}

Future<void> updateStudent() async {
  if (documentId == null) return;

  try {
    await students.doc(documentId).update({
      'name': nameController.text.trim(),
      'department': departmentController.text.trim(),
      'email': emailController.text.trim(),
    });

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Student updated successfully!'),
      ),
    );

    setState(() {});
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Update failed: $e')),
    );
  }
}
```

```
@override
void dispose() {
  registerController.dispose();
  nameController.dispose();
  departmentController.dispose();
  emailController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Update Student Information'),
      backgroundColor: Colors.indigo,
      foregroundColor: Colors.white,
    ),
    body: Padding(
      padding: const EdgeInsets.all(20),
      child: SingleChildScrollView(
        child: Column(
          children: [
            TextField(
              controller: registerController,
              decoration: const InputDecoration(
                labelText: 'Enter Register Number',
                border: OutlineInputBorder(),
                prefixIcon: Icon(Icons.search),
              ),
            ),

            const SizedBox(height: 12),

            ElevatedButton.icon(
              onPressed: searchStudent,
              icon: const Icon(Icons.search),
              label: const Text('Find Student'),
            ),

            const SizedBox(height: 20),

            if (isLoading)
              const CircularProgressIndicator(),

            if (documentId != null) ...[
              const Text(
                'Edit Student Details',
                style: TextStyle(
                  fontSize: 20,
                  fontWeight: FontWeight.bold,
                ),
              ),

              const SizedBox(height: 15),
```

```
            TextField(
              controller: nameController,
              decoration: const InputDecoration(
                labelText: 'Student Name',
                border: OutlineInputBorder(),
              ),
            ),

            const SizedBox(height: 12),

            TextField(
              controller: departmentController,
              decoration: const InputDecoration(
                labelText: 'Department',
                border: OutlineInputBorder(),
              ),
            ),

            const SizedBox(height: 12),

            TextField(
              controller: emailController,
              decoration: const InputDecoration(
                labelText: 'Email',
                border: OutlineInputBorder(),
              ),
            ),

            const SizedBox(height: 20),

            ElevatedButton.icon(
              onPressed: updateStudent,
              icon: const Icon(Icons.update),
              label: const Text('Update Student'),
            ),
          ],
        ],
      ),
    ),
  ),
 );
 }
}
```

Output Screenshots

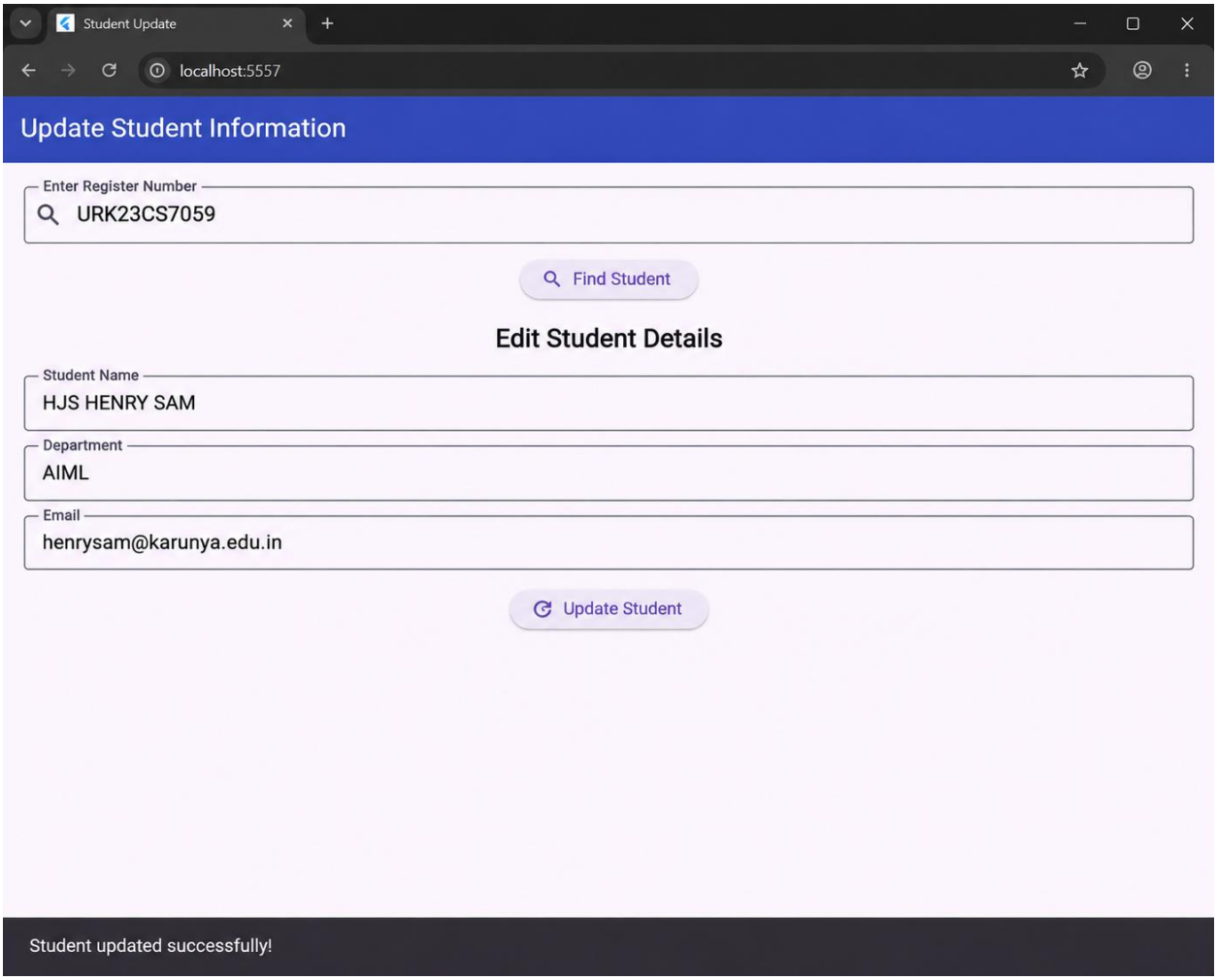


Figure 1: Student Update application displaying the retrieved student details and the successful update confirmation message.

// Output of the program

Result

Thus, the Flutter application for updating student information in Google Cloud Firestore was successfully developed and executed. The application retrieved an existing student record using the register number, displayed the details in editable fields and updated the existing Firestore document using the update() method. The successful update confirmation was displayed on the application screen.

Ex. No. 10	Manual User Login Authentication Using Google Firestore Database
Date of Exercise	01 November 2026
Github Code Link(Keep as public)	https://github.com/henrysam/FlutterLaboratory

Aim

To develop a Flutter application that allows users to register and log in using email and password credentials stored in Google Cloud Firestore, and displays a dashboard after successful authentication.

Description

In this experiment, a Flutter application named Secure Login is developed to demonstrate manual user registration and login using Google Cloud Firestore. The application provides registration and login interfaces with email and password input fields. During registration, the entered email address is converted to lowercase and used to generate a document identifier. A random salt is generated, and the password is combined with the salt and hashed using the SHA-256 algorithm provided by the crypto package. The email address, salt, password hash and server timestamp are stored in the Firestore 'users' collection. During login, the application retrieves the corresponding user document, reads the stored salt and password hash, hashes the entered password using the same salt, and compares the result with the stored hash. If the values match, the application navigates to a dashboard displaying the logged-in email address. Otherwise, an appropriate error message is displayed. The interface also supports switching between registration and login, showing or hiding the password, and logging out to return to the login screen. Firebase is initialized using Firebase.initializeApp() with the configuration generated by the FlutterFire CLI. The application was executed in Google Chrome and tested with registration and successful login. This is a classroom demonstration of manual credential checking; production applications should use a dedicated authentication service.

Program

```
import 'dart:convert';
import 'dart:math';

import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:crypto/crypto.dart';
```

```
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Secure Login',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.indigo,
        ),
        useMaterial3: true,
      ),
      home: const AuthPage(),
    );
  }
}

class AuthPage extends StatefulWidget {
  const AuthPage({super.key});

  @override
  State<AuthPage> createState() => _AuthPageState();
}

class _AuthPageState extends State<AuthPage> {
  final emailController = TextEditingController();
  final passwordController = TextEditingController();

  final users = FirebaseFirestore.instance.collection('users');

  bool isLogin = true;
  bool isLoading = false;
  bool obscurePassword = true;

  String getUserId(String email) {
    return sha256.convert(utf8.encode(email.toLowerCase())).toString();
  }

  String generateSalt() {
    final random = Random.secure();
    final bytes = List<int>.generate(
```

```
    16,
    (_) => random.nextInt(256),
  );
  return base64Encode(bytes);
}

String hashPassword(String password, String salt) {
  return sha256
      .convert(utf8.encode('$salt$password'))
      .toString();
}

Future<void> authenticate() async {
  final email = emailController.text.trim().toLowerCase();
  final password = passwordController.text;

  if (email.isEmpty || password.isEmpty) {
    showMessage('Please enter email and password');
    return;
  }

  if (!email.contains('@') || !email.contains('.')) {
    showMessage('Please enter a valid email address');
    return;
  }

  if (password.length < 6) {
    showMessage('Password must contain at least 6 characters');
    return;
  }

  setState(() => isLoading = true);

  try {
    final userId = getUserId(email);
    final userDoc = await users.doc(userId).get();

    if (isLogin) {
      if (!userDoc.exists) {
        showMessage('Account not found. Please register.');
      } else {
        final data = userDoc.data()!;
        final salt = data['salt'] as String;
        final storedHash = data['passwordHash'] as String;
        final enteredHash = hashPassword(password, salt);

        if (enteredHash == storedHash) {
          if (!mounted) return;

          Navigator.pushReplacement(
            context,
            MaterialPageRoute(
              builder: (_) => DashboardPage(email: email),
            ),
          );
        } else {
          showMessage('Incorrect password');
```

```
      }
     }
    } else {
     if (userDoc.exists) {
      showMessage('Account already exists. Please login.');
     } else {
      final salt = generateSalt();
      final passwordHash = hashPassword(password, salt);

      await users.doc(userId).set({
       'email': email,
       'salt': salt,
       'passwordHash': passwordHash,
       'createdAt': FieldValue.serverTimestamp(),
      });

      showMessage('Registration successful! Please login.');

      setState(() {
       isLogin = true;
       passwordController.clear();
      });
     }
    }
  } catch (e) {
    showMessage('Authentication failed: $e');
  } finally {
    if (mounted) {
     setState(() => isLoading = false);
    }
  }
}

void showMessage(String message) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text(message)),
  );
}

@override
void dispose() {
  emailController.dispose();
  passwordController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Container(
      width: double.infinity,
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          colors: [
            Color(0xFF3949AB),
            Color(0xFF7986CB),
            Color(0xFFF5F5FF),
          ],
          begin: Alignment.topCenter,
```

```
      end: Alignment.bottomCenter,
    ),
  ),
  child: Center(
    child: SingleChildScrollView(
      padding: const EdgeInsets.all(24),
      child: Card(
        elevation: 8,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(24),
        ),
        child: Padding(
          padding: const EdgeInsets.all(28),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const CircleAvatar(
                radius: 35,
                backgroundColor: Color(0xFFE8EAF6),
                child: Icon(
                  Icons.lock_outline,
                  size: 38,
                  color: Colors.indigo,
                ),
              ),
              const SizedBox(height: 20),
              Text(
                isLogin ? 'Welcome Back!' : 'Create Account',
                style: const TextStyle(
                  fontSize: 26,
                  fontWeight: FontWeight.bold,
                ),
              ),
              const SizedBox(height: 8),
              Text(
                isLogin
                    ? 'Login to continue'
                    : 'Register to get started',
                style: TextStyle(color: Colors.grey[600]),
              ),
              const SizedBox(height: 30),
              TextField(
                controller: emailController,
                keyboardType: TextInputType.emailAddress,
                decoration: InputDecoration(
                  labelText: 'Email Address',
                  prefixIcon: const Icon(Icons.email_outlined),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
              ),
              const SizedBox(height: 18),
              TextField(
                controller: passwordController,
                obscureText: obscurePassword,
                decoration: InputDecoration(
                  labelText: 'Password',
                  prefixIcon: const Icon(Icons.lock_outline),
                  suffixIcon: IconButton(
                    icon: Icon(
```

```
                obscurePassword
                    ? Icons.visibility_off
                    : Icons.visibility,
              ),
              onPressed: () {
                setState(() {
                  obscurePassword = !obscurePassword;
                });
              },
            ),
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(12),
            ),
          ),
        ),
        const SizedBox(height: 25),
        SizedBox(
          width: double.infinity,
          height: 50,
          child: ElevatedButton(
            onPressed: isLoading ? null : authenticate,
            style: ElevatedButton.styleFrom(
              backgroundColor: Colors.indigo,
              foregroundColor: Colors.white,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(12),
              ),
            ),
            child: isLoading
                ? const CircularProgressIndicator(
                    color: Colors.white,
                  )
                : Text(
                    isLogin ? 'LOGIN' : 'REGISTER',
                    style: const TextStyle(
                      fontWeight: FontWeight.bold,
                    ),
                  ),
          ),
        ),
        const SizedBox(height: 15),
        TextButton(
          onPressed: isLoading
              ? null
              : () {
                  setState(() {
                    isLogin = !isLogin;
                    passwordController.clear();
                  });
                },
          child: Text(
            isLogin
                ? "Don't have an account? Register"
                : 'Already have an account? Login',
          ),
        ),
      ],
    ),
  ),
),
```

```
      ),
    ),
  );
 }
}

class DashboardPage extends StatelessWidget {
 final String email;

 const DashboardPage({super.key, required this.email});

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(
    title: const Text('Dashboard'),
    backgroundColor: Colors.indigo,
    foregroundColor: Colors.white,
    actions: [
     IconButton(
      icon: const Icon(Icons.logout),
      tooltip: 'Logout',
      onPressed: () {
       Navigator.pushReplacement(
        context,
        MaterialPageRoute(
         builder: (_) => const AuthPage(),
        ),
       );
      },
     ),
    ],
   ),
   body: Center(
    child: Padding(
     padding: const EdgeInsets.all(24),
     child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
       const CircleAvatar(
        radius: 45,
        child: Icon(Icons.person, size: 50),
       ),
       const SizedBox(height: 20),
       const Text(
        'Login Successful!',
        style: TextStyle(
         fontSize: 26,
         fontWeight: FontWeight.bold,
        ),
       ),
       const SizedBox(height: 12),
       Text(
        'Welcome, $email',
        textAlign: TextAlign.center,
        style: const TextStyle(fontSize: 16),
       ),
       const SizedBox(height: 25),
       ElevatedButton.icon(
        onPressed: () {
```

```
                Navigator.pushReplacement(
                  context,
                  MaterialPageRoute(
                    builder: (_) => const AuthPage(),
                  ),
                );
              },
              icon: const Icon(Icons.logout),
              label: const Text('Logout'),
            ),
          ],
        ),
      ),
    ),
  );
 }
}
```

Output Screenshots

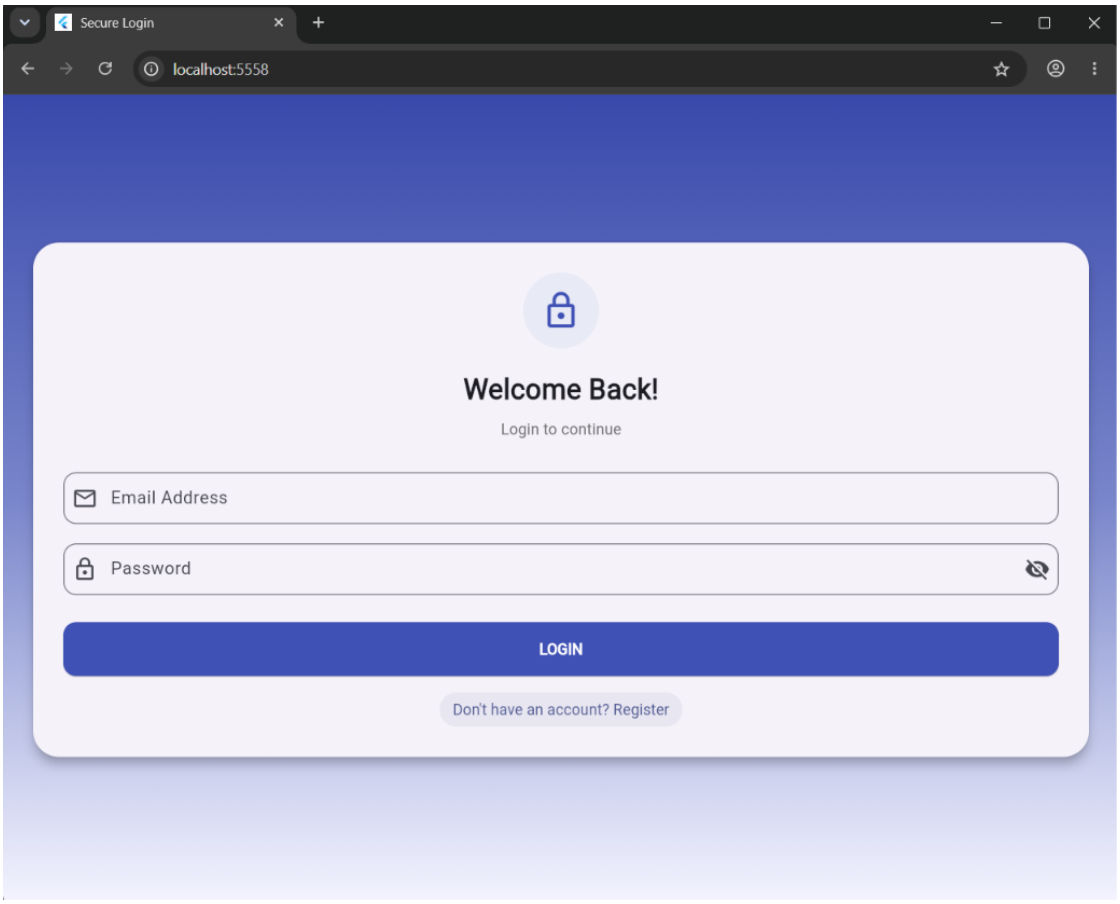


Figure 1: Secure Login application displaying the user login screen.

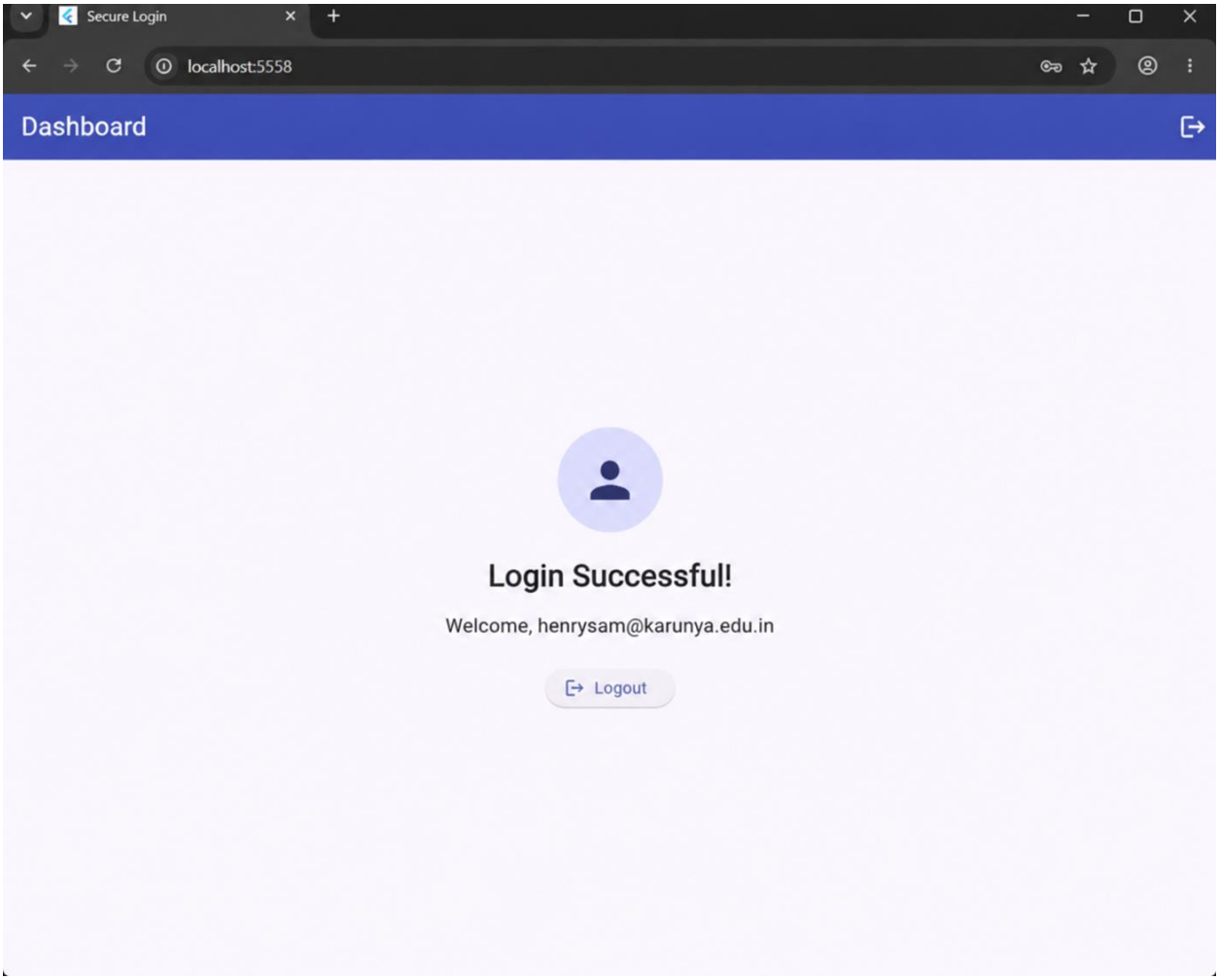


Figure 2: Dashboard displayed after successful user authentication.

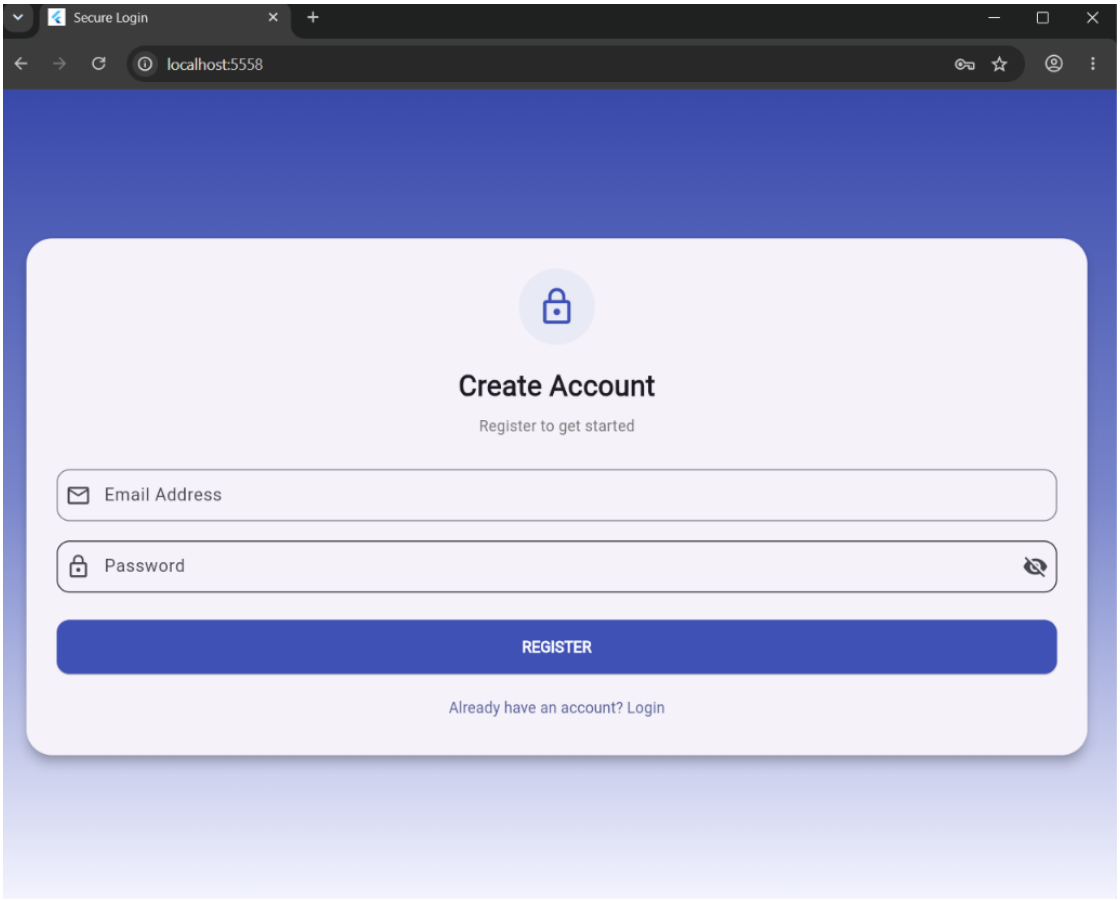


Figure 3: User registration screen for creating a new account.

// Output of the program

Result

Thus, the Flutter application for manual user login authentication using Google Cloud Firestore was successfully developed and executed. The application provided registration and login screens, stored account credentials as salted password hashes in the Firestore 'users' collection, verified the entered password during login, and displayed the dashboard after successful authentication. The registration, login and dashboard screens were tested successfully.